TASK 14

KUBERNETES SCALING AND ROLLING UPDATES

Name: Monu Ajay Rai

Student ID: 5783524

Roll no: 27

Subject: Deep Learning from Production to Deployment

GitHub URL:

https://github.com/monu8097/Task14_Kubernetes_Scaling_Rolling_Updates.git

Table of Contents

1. Introduction
2. Objective
3. Project Overview
4. Technologies and Tools Used
5. Kubernetes Environment Setup
6. Project Structure
7. Initial Deployment Verification
8. Application Scaling
9. Scaling from 2 to 3 Replicas
10. Pod and Deployment Monitoring
11. Resource Utilization Monitoring
12. Rolling Update Configuration
13. Rolling Update Implementation
14. ReplicaSet Verification
15. Rollout History and Status
16. Service Verification
17. Application Availability Testing
18. Streamlit Application Verification
19. Results, Observations and Analysis
20. Conclusion

1. Introduction

1.1 Background

Modern machine learning systems are rarely judged only by the accuracy of their models. Once a model leaves the notebook and becomes a service, a different set of questions becomes important: can the service handle more load when demand grows, can it be updated without taking users offline, and can its health and resource consumption be observed while it runs? Container orchestration platforms exist to answer exactly these questions, and Kubernetes has become the most widely used platform for the purpose.

This report documents Task 14, **Kubernetes Scaling and Rolling Updates**. The work builds on a containerized deep learning application that classifies images using a Convolutional Neural Network (CNN) trained on the CIFAR-10 dataset. The application has two cooperating parts: a Flask-based REST API that loads the trained model and returns predictions, and a Streamlit-based web front end through which a user uploads an image and sees the result. Earlier stages of the project packaged these two parts as Docker containers and deployed them to a Kubernetes cluster; the Docker Compose file and the Streamlit footer in the submitted files refer to Task 11 (full application containerization) and the image names in the Kubernetes manifests carry the prefix `task12`. Task 14 takes that existing deployment and exercises the operational features of Kubernetes on it.

1.2 Problem Statement

A single running copy of a prediction service is a single point of limitation. If the copy is busy, further requests wait; if the copy has to be replaced during an upgrade, there is a window in which no copy is available. The problem addressed in this task is therefore twofold. First, the number of running copies (replicas) of the prediction API must be changed on demand and the change must be verified. Second, a modification to the deployment must be rolled out in a controlled way so that the application stays reachable and functional while old pods are replaced by new ones.

1.3 Scope of the Work

The scope of this task is intentionally operational rather than algorithmic. The CNN model, the Flask code and the Streamlit code are reused without functional change. The work carried out and documented here consists of the following activities:

- Starting and verifying a local Kubernetes cluster using Minikube with the Docker driver.
- Verifying the existing Flask API and Streamlit deployments before any change is made.
- Scaling the Flask API deployment from two replicas to three replicas with `kubectl scale`.
- Enabling the Kubernetes Metrics Server and measuring CPU and memory consumption of every pod with `kubectl top pods`.
- Configuring the `RollingUpdate` strategy with `maxUnavailable: 1` and `maxSurge: 1`, and triggering a rolling update by changing the pod template (the `TASK14_VERSION` environment variable).
- Verifying the resulting ReplicaSets, the rollout history, the rollout status and the final state of pods.
- Testing the availability of the Flask service from inside the cluster, before and after the update.
- Testing the complete user path through the Streamlit front end by predicting the class of a frog image after the update.

1.4 Evidence Used in This Report

Every observation in this report is taken from the files supplied with the task: the Kubernetes manifests, the Dockerfiles, the application source code, the README file, the trained model file and a set of seventeen terminal and browser screenshots, together with one architecture diagram and the frog image used for the final prediction test. No figures or numbers have been added from outside these files. Where a value shown in a screenshot differs from the value quoted in the README (for example, the exact memory figures of individual pods), the value visible in the screenshot is treated as authoritative.

1.5 Organization of the Report

Sections 2 to 4 describe the objective, the overall project and the technologies. Section 5 and Section 6 describe the cluster and the project files. Sections 7 to 11 cover the initial deployment, the scaling operation and the monitoring of pods and resources. Sections 12 to 15 cover the rolling update from configuration to history. Sections 16 to 18 cover services and availability testing, including the end-to-end test through the Streamlit application. Section 19 analyses the results against the evaluation criteria of the task and Section 20 concludes the report.

2. Objective

2.1 Task Objective

The objective stated for Task 14 is **to implement scalability and deployment update strategies using Kubernetes**. In practical terms, this means demonstrating that the application can be made larger or smaller by changing a replica count, and that a change to the deployment can be rolled out gradually with the application remaining available throughout.

2.2 Approach Prescribed by the Task

The task prescribes five lines of approach. Each of them is carried out in this report and is tied to specific evidence:

- **Scale application replicas** – the Flask API deployment is scaled from two to three replicas (Sections 8 and 9).
- **Perform rolling updates** – a pod-template change is rolled out using the `RollingUpdate` strategy (Sections 12 to 15).
- **Monitor deployment status** – deployments, pods, ReplicaSets and rollout status are inspected after every change (Sections 10, 14 and 15).
- **Validate application availability** – the Flask service is called from a temporary pod inside the cluster and the Streamlit application is used from the browser (Sections 17 and 18).
- **Analyze resource utilization** – the Metrics Server is enabled and pod CPU and memory usage is recorded before and after the update (Section 11).

2.3 Deliverables

The task asks for three deliverables: scaling documentation, monitoring screenshots and performance observations. This report is the scaling documentation; the seventeen screenshots are embedded as figures and are listed in the project structure; and the performance observations are consolidated in Sections 11 and 19.

2.4 Evaluation Criteria

The work is to be judged on three criteria: **scaling effectiveness**, **rolling update implementation** and **system stability**. The report is organized so that each criterion can be traced to a specific group of sections and figures, as shown in the table below.

Evaluation criterion	What is demonstrated	Where documented
Scaling effectiveness	Flask deployment moves from 2/2 to 3/3 ready replicas; new pod is scheduled and Running; rollout reports success.	Sections 8, 9, 10
Rolling update implementation	RollingUpdate strategy with maxUnavailable 1 and maxSurge 1; new ReplicaSet created; old ReplicaSets scaled to zero; three revisions recorded.	Sections 12, 13, 14, 15
System stability	All pods Running and Ready after changes; stable and low resource usage; service and Streamlit prediction working after the update.	Sections 11, 16, 17, 18, 19

2.5 Mapping of Approach Items to Evidence

Approach item	Command or configuration	Evidence figure
Scale replicas	kubectl scale deployment flask-api --replicas=3	Figure 4, Figure 5
Perform rolling updates	kubectl apply -f flask-deployment.yaml	Figure 9, Figure 10
Monitor deployment status	kubectl get deployments / pods / rs; kubectl rollout status	Figure 6, Figure 18
Validate availability	kubectl run flask-test ... curl -s http://flask-api-service:5000/	Figure 13, Figure 14, Figure 17
Analyze resource utilization	minikube addons enable metrics-server; kubectl top pods	Figure 7, Figure 8, Figure 19

3. Project Overview

3.1 Application Summary

The application is a CIFAR-10 image classification system. CIFAR-10 contains 60,000 colour images of size 32 × 32 pixels in ten classes; the Streamlit dashboard of the project reports 50,000 training images and 10,000 test images. The ten classes are Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck. A user uploads an image, the front end sends it to the back end, the back end resizes it to the input size of the model, runs the CNN and returns the predicted class together with a confidence percentage.

3.2 System Architecture

The architecture of the Task 14 environment is shown in Figure 1. It separates two concerns. The right-hand side shows the application path: an image uploader interacts with the Streamlit user interface defined in `app.py`, which posts the image to the Flask API defined in `flask_api.py`, which predicts using the CIFAR-10 CNN stored in `cifar10_cnn.keras`. The left and centre of the diagram show the operational path: a cluster operator checks resource usage through the Metrics Server, scales replicas, verifies the services and applies updates, and these actions act on the Streamlit and Flask services and deployments that run inside the Kubernetes cluster.

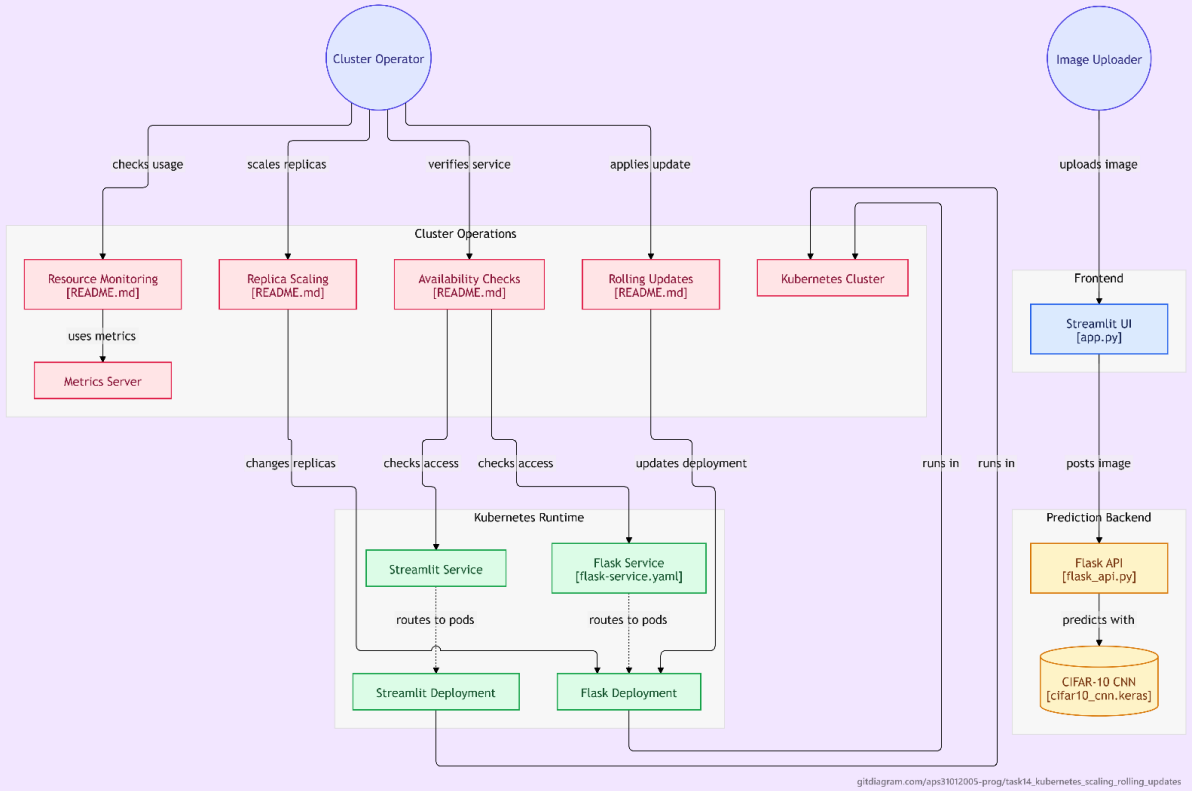


Figure 1: *Task 14 architecture: cluster operations on the left and the application request path on the right*

3.3 Components

Component	Role in the system	Kubernetes object
Streamlit front end (`app.py`)	Web interface for uploading an image, showing dataset and model information, checking API status and displaying the prediction.	Deployment `streamlit` (2 replicas) and NodePort Service `streamlit-service`
Flask API (`flask_api.py`)	Loads `model/cifar10_cnn.keras` once at start-up and exposes a health endpoint and a prediction endpoint.	Deployment `flask-api` (2 then 3 replicas) and ClusterIP Service `flask-api-service`
CNN model	Sequential CNN trained on CIFAR-10 and stored in Keras format; packaged inside the Flask image.	Part of the Flask container image
Metrics Server	Collects pod CPU and memory usage so that `kubectl top` can report it.	Add-on in the `kube-system` namespace
Minikube cluster	Single-node local Kubernetes cluster that hosts all objects.	Node `minikube`

3.4 API Endpoints

Endpoint	Method	Purpose	Behaviour seen in the code
`/`	GET	Health check	Returns JSON containing a running message, the name of the prediction endpoint and its method.
`/predict`	POST	Image classification	Expects a multipart field named `image`; returns `predicted_class` and `confidence`, or an error with HTTP 400 (missing image) or 500 (processing failure).

3.5 Request Flow of a Prediction

1. The user selects a JPG, JPEG or PNG file in the Streamlit page and presses **Predict Image**.
2. Streamlit converts the file to RGB for display and posts the original file to `http://flask-api-service:5000/predict` as a multipart form field named `image`, with a timeout of 60 seconds.
3. Inside the cluster, the name `flask-api-service` is resolved to the ClusterIP of the Flask service, and the request is forwarded to one of the ready Flask pods.
4. The Flask pod opens the image, converts it to RGB, resizes it to 32 × 32 pixels, scales pixel values to the range 0–1 and adds a batch dimension.
5. The model predicts a probability for each of the ten classes; the class with the highest probability is selected and its probability is multiplied by 100 and rounded to two decimals.
6. The JSON result is returned to Streamlit, which shows the class, the confidence, a progress bar and the raw JSON.

3.6 The CNN Model

The model file supplied with the task is a Keras `Sequential` model. Its configuration, read from the saved file, consists of an input of shape 32 × 32 × 3, a 3 × 3 convolution with 32 filters and ReLU activation, a 2 × 2 max-pooling layer, a second 3 × 3 convolution with 64 filters and ReLU activation, another 2 × 2 max-pooling layer, a flatten layer, a dense layer of 128 units with ReLU activation, a dropout layer with rate 0.5 and a final dense layer of 10 units with softmax activation. The Streamlit page describes the training set-up as the Adam optimizer with sparse categorical crossentropy loss, ten epochs and a test accuracy of 72.4%.

Layer	Configuration	Output shape	Parameters
Input	32 × 32 × 3 image	(32, 32, 3)	0
Conv2D	32 filters, 3 × 3, ReLU	(30, 30, 32)	896
MaxPooling2D	2 × 2	(15, 15, 32)	0
Conv2D	64 filters, 3 × 3, ReLU	(13, 13, 64)	18,496
MaxPooling2D	2 × 2	(6, 6, 64)	0
Flatten	–	(2304)	0
Dense	128 units, ReLU	(128)	295,040
Dropout	rate 0.5 (training only)	(128)	0
Dense	10 units, softmax	(10)	1,290

The output shapes and parameter counts in the table are derived from the layer configuration by standard convolution arithmetic (no padding, stride 1), giving a total of 315,722 trainable parameters. The model is therefore small; as discussed in Section 11, the memory footprint of about 250 MiB observed for each Flask pod is much larger than the model weights alone and is dominated by the Python and TensorFlow runtime that every replica must load.

3.7 Why This Application Suits Scaling Experiments

The Flask API is stateless: it does not store uploaded images, sessions or results. Every request is independent, so any replica can serve any request. This property is what allows Kubernetes to add or remove replicas freely and to replace them during a rolling update. The front end locates the API through a stable service name instead of a pod address, so the front end is unaffected when individual API pods are created or destroyed.

4. Technologies and Tools Used

4.1 Overview

The project combines application technologies (Python, Flask, Streamlit, TensorFlow), packaging technologies (Docker) and orchestration technologies (Kubernetes, Minikube, kubectl, Metrics Server). The table below lists each tool, the purpose it serves in Task 14 and where it is visible in the supplied files.

Technology	Purpose in Task 14	Where it appears
Python 3.11	Runtime for both containers.	Base image `python:3.11-slim` in both Dockerfiles
Flask	REST API serving predictions.	`flask_api.py`, `requirements.txt`
Streamlit	Interactive front end.	`app.py`, Dockerfile.streamlit
TensorFlow / Keras	Loading and running the CNN.	`flask_api.py`, `cifar10_cnn.keras`
NumPy and Pillow	Image conversion, resizing and array handling.	`flask_api.py`, `app.py`
Requests	HTTP calls from Streamlit to Flask.	`app.py`
Docker	Container images of both services.	Dockerfile.flask, Dockerfile.streamlit, .dockerignore
Docker Compose	Local multi-container run used in an earlier task.	`docker-compose.yml`
Kubernetes	Orchestration: deployments, replica sets, services, rollouts.	All four YAML manifests
Minikube	Local single-node cluster using the Docker driver.	Figure 2
kubectl	Command-line client for every operation.	All terminal figures
Metrics Server	Pod CPU and memory metrics.	Figure 7
curlimages/curl	Temporary pod used to call the Flask service from inside the cluster.	Figure 13, Figure 14
PowerShell on Windows 11	Terminal used to run the commands.	Prompt visible in the terminal figures

4.2 Application Technologies

Flask is a lightweight Python web framework. In this project it defines two routes and loads the Keras model once, at import time, into a module-level variable so that the model is not reloaded on every request. **Streamlit** turns a Python script into a web page; the script `app.py`

builds the dashboard, the uploader and the prediction display. **TensorFlow with Keras** provides `tf.keras.models.load_model`, which restores the architecture and weights from the `.keras` file. **Pillow** and **NumPy** convert the uploaded image into the normalized array that the model expects, and the **requests** library carries the image from Streamlit to Flask.

4.3 Packaging Technologies

Both services are built from the slim Python 3.11 image. The Flask image copies `requirements.txt`, installs the dependencies with `pip install --no-cache-dir`, copies `flask_api.py` and the `model` directory and starts the API on port 5000. The Streamlit image performs the same dependency installation, copies `app.py` and the `pages` directory and starts Streamlit on address 0.0.0.0 and port 8501. The `.dockerignore` file keeps Python cache files, version-control data, editor settings, the screenshots folder and the README file out of the build context so that images remain lean.

4.4 Orchestration Technologies

Kubernetes is the platform that keeps the declared state of the application (how many replicas, which image, which resources, which update strategy) equal to the actual state of the cluster. **Minikube** creates a one-node cluster on a developer machine; in this task it ran on Windows 11 using the Docker driver. **kubectl** is the client that talks to the Kubernetes API server. The **Metrics Server** is a cluster add-on that aggregates resource usage from the kubelets and exposes it through the metrics API, which is what `kubectl top` reads.

4.5 Why These Choices Fit the Task

- A local single-node cluster is sufficient to demonstrate replica management, rolling updates and metrics, because all of these are control-plane features that do not depend on multiple machines.
- Using `imagePullPolicy: Never` with locally built images removes the need for an external container registry.
- Using a short-lived curl pod for testing avoids changing the application and verifies the service from the same network position as the Streamlit pods.

5. Kubernetes Environment Setup

5.1 Choice of Cluster

A local Kubernetes cluster was created with Minikube. Minikube runs a complete Kubernetes control plane and a worker role on a single node, which makes it suitable for learning and for demonstrating features such as scaling and rolling updates without the cost or complexity of a multi-node cluster. The cluster was started with the Docker driver, which means that the Kubernetes node itself runs as a Docker container on the host machine.

5.2 Starting the Cluster

The cluster was started from the project folder using the following command:

```
minikube start --driver=docker
```

The terminal output is shown in Figure 2. The first lines show that Minikube version 1.39.0 was running on Windows 11 and that the Docker driver was chosen "based on existing profile", which means that a cluster named `minikube` had been created earlier and was being restarted rather than created from scratch.

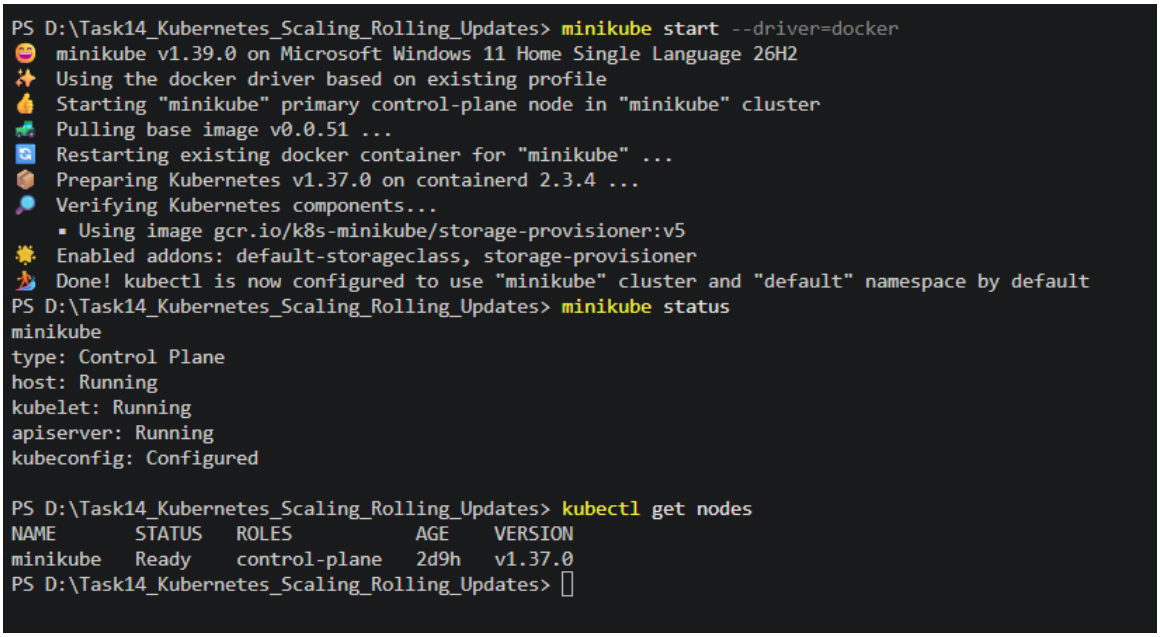


Figure 2: *Starting Minikube with the Docker driver, checking its status and listing the node*

5.3 Reading the Start-up Output

Output line (summarized)	Meaning
Using the docker driver based on existing profile	The existing `minikube` profile is reused; no new cluster is created.
Starting "minikube" primary control-plane node	The single node that hosts the control plane is started.
Restarting existing docker container for "minikube"	The node container created earlier is started again, so the state stored in the cluster (deployments, services) is preserved.
Preparing Kubernetes v1.37.0 on containerd 2.3.4	The Kubernetes version and the container runtime used inside the node.
Verifying Kubernetes components	Minikube waits for the control-plane components to become healthy.
Using image gcr.io/k8s-minikube/storage-provisioner:v5	The storage provisioner add-on image.
Enabled addons: default-storageclass, storage-provisioner	Add-ons enabled at this point; the Metrics Server is enabled later (Section 11).
Done! kubectl is now configured to use "minikube" cluster and "default" namespace	The kubeconfig context points to this cluster; all later commands act on the `default` namespace.

The restart of an existing container is relevant to later observations. In the pod listings the application pods show one restart each, with a "time ago" value that is small compared with the pod age. This is consistent with the application containers having been restarted when the node was brought back up by `minikube start`; the pods themselves were created more than a day earlier.

5.4 Verifying Cluster Health

Two commands confirmed that the cluster was healthy before any work began:

```
minikube status
kubectl get nodes
```

`minikube status` reported the node type as Control Plane and the host, kubelet and apiserver as Running, with the kubeconfig Configured. `kubectl get nodes` listed one node named `minikube` with status **Ready**, role `control-plane`, age 2d9h and version v1.37.0 (Figure 2).

Item	Observed value
Host operating system (as printed by Minikube)	Windows 11 Home Single Language
Minikube version	v1.39.0

Item	Observed value
Driver	docker
Kubernetes version	v1.37.0
Container runtime	containerd 2.3.4
Node name / role / status	minikube / control-plane / Ready
Node age	2 days 9 hours
Namespace used	default
Working folder (from prompt)	D:\Task14_Kubernetes_Scaling_Rolling_Updates

5.5 What the Control Plane Provides

The control plane components that matter for this task are the API server, which receives every kubectl command; the controller manager, which contains the Deployment and ReplicaSet controllers that create and delete pods to match the declared replica count; the scheduler, which assigns new pods to a node; and the kubelet on the node, which starts the containers. Because the cluster has a single node, the scheduler has only one choice of node, and all pods in the listings show the node `minikube`. This has an important consequence discussed in Section 19: scaling in this environment increases the number of serving processes but cannot protect against the loss of the node itself.

5.6 Reproducing the Setup

1. Install Docker, Minikube and kubectl on the workstation.
2. Run `minikube start --driver=docker` and wait for the "Done!" message.
3. Run `minikube status` and confirm that host, kubelet and apiserver are Running.
4. Run `kubectl get nodes` and confirm that the node is Ready.
5. Make the application images available to the node so that `imagePullPolicy: Never` can find `task12-flask-api:latest` and `task12-streamlit:latest`.
6. Apply the manifests (deployments and services) with `kubectl apply -f`.

6. Project Structure

6.1 Folder Layout

The submission folder is named `Task14_Kubernetes_Scaling_Rolling_Updates`. It contains the trained model, the screenshots, the application source code, the container build files and the Kubernetes manifests:

Project tree

```
Task14_Kubernetes_Scaling_Rolling_Updates/
|
+-- model/
|   +-- cifar10_cnn.keras
|
+-- screenshots/
|   +-- 01_task14_cluster_status.png
|   +-- 02_task14_initial_deployment_status.png
|   +-- 03_flask_api_scaled_to_3_replicas.png
|   +-- 04_flask_scaling_rollout_status.png
|   +-- 05_flask_api_availability_after_scaling.png
|   +-- 06_task14_resource_utilization.png
|   +-- 07_task14_rolling_update_configuration.png
|   +-- 08_task14_rolling_update_replicaset.png
|   +-- 09_task14_rollout_history.png
|   +-- 10_task14_final_deployment_pod_status.png
|   +-- 11_task14_resource_utilization_after_update.png
|   +-- 12_task14_api_availability_after_update.png
|   +-- 13_task14_services_verification.png
|   +-- 14_task14_streamlit_application.png
|   +-- 15_task14_frog_prediction_after_update.png
|   +-- 16_task14_final_kubernetes_verification.png
|   +-- 17_task14_final_resource_and_rollout_status.png
|
+-- .dockerignore
+-- app.py
+-- docker-compose.yml
+-- Dockerfile.flask
+-- Dockerfile.streamlit
+-- flask_api.py
+-- flask-deployment.yaml
+-- flask-service.yaml
+-- README.md
+-- requirements.txt
+-- streamlit-deployment.yaml
+-- streamlit-service.yaml
```

6.2 Purpose of Each File

File	Category	Purpose
`model/cifar10_cnn.keras`	Model	Trained CIFAR-10 CNN (about 3.7 MiB on disk) loaded by the Flask API.
`flask_api.py`	Application	Flask REST API with health and prediction endpoints.
`app.py`	Application	Streamlit dashboard and image upload interface.
`requirements.txt`	Application	Python dependencies shared by both images.
`Dockerfile.flask`	Container	Build instructions for the Flask image.

File	Category	Purpose
`Dockerfile.streamlit`	Container	Build instructions for the Streamlit image.
`.dockerignore`	Container	Files excluded from the Docker build context.
`docker-compose.yml`	Container	Two-container local run from the earlier containerization task.
`flask-deployment.yaml`	Kubernetes	Flask Deployment: 3 replicas, RollingUpdate strategy, resources, TASK14_VERSION variable.
`flask-service.yaml`	Kubernetes	ClusterIP service `flask-api-service` on port 5000.
`streamlit-deployment.yaml`	Kubernetes	Streamlit Deployment: 2 replicas with resource requests and limits.
`streamlit-service.yaml`	Kubernetes	NodePort service `streamlit-service` on port 8501.
`README.md`	Documentation	Short description of the task, commands and results.
`screenshots/`	Evidence	Seventeen numbered screenshots used as figures in this report.

One detail is worth recording. `Dockerfile.streamlit` contains the instruction `COPY pages ./pages`, and the browser screenshot in Section 18 shows a second page named "Image Prediction" in the sidebar. The `pages` directory is not part of the folder listing given for Task 14, so it is assumed to have been present in the build context when the image was built.

6.3 Container Build Files

Listing 6.1 – Dockerfile.flask

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY flask_api.py .
COPY model ./model

EXPOSE 5000

CMD ["python", "flask_api.py"]
```

The Flask image installs dependencies before copying source code. Because Docker caches each layer, a change to `flask_api.py` alone does not force the slow TensorFlow installation to run again. The model directory is copied into the image, so every Flask replica carries its own copy of the model and needs no shared volume.

Listing 6.2 – Dockerfile.streamlit

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .
COPY pages ./pages

EXPOSE 8501

CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

The Streamlit image is built the same way, with the start command binding Streamlit to all network interfaces (`--server.address=0.0.0.0`) so that traffic arriving through the pod IP and the service can reach it.

Listing 6.3 – .dockerignore

```
__pycache__
*.pyc
*.pyo
*.pyd
.git
.gitignore
.vscode
screenshots
screenshots9
README.md
```

Listing 6.4 – requirements.txt

```
flask
streamlit
requests
tensorflow
numpy
pillow
```

The requirements file lists six packages without version pins. This keeps the file short, but it means that rebuilding the images at a later date could install newer library versions. Section 19 returns to this point as a recommendation.

Listing 6.5 – docker-compose.yml (earlier containerization task)

```
services:

  flask-api:
    build:
      context: .
      dockerfile: Dockerfile.flask
```

```
    container_name: task11-flask-api

    ports:
      - "5000:5000"

    networks:
      - task11-network


  streamlit:
    build:
      context: .
      dockerfile: Dockerfile.streamlit

    container_name: task11-streamlit

    ports:
      - "8501:8501"

    depends_on:
      - flask-api

    networks:
      - task11-network


networks:
  task11-network:
    driver: bridge
```

The Compose file builds the same two images and connects them through a bridge network. It is included for completeness; Task 14 itself does not use Compose, because Kubernetes takes over the roles of networking, scheduling and replica management.

6.4 Application Source Code

Listing 6.6 – flask_api.py

```
from flask import Flask, request, jsonify
import tensorflow as tf
import numpy as np
from PIL import Image
import io

# ============================================
# Flask Application
# ============================================

app = Flask(__name__)



# ============================================
# CIFAR-10 Class Names
# ============================================

class_names = [
    "Airplane",
    "Automobile",
```

```
    "Bird",
    "Cat",
    "Deer",
    "Dog",
    "Frog",
    "Horse",
    "Ship",
    "Truck"
]



# ========================================
# Load Trained CNN Model
# ========================================

MODEL_PATH = "model/cifar10_cnn.keras"

model = tf.keras.models.load_model(MODEL_PATH)



# ========================================
# Home / Health Check Endpoint
# ========================================

@app.route("/", methods=["GET"])
def home():
    return jsonify({
        "message": "CIFAR-10 Flask Prediction API is running",
        "endpoint": "/predict",
        "method": "POST"
    })



# ========================================
# Prediction Endpoint
# ========================================

@app.route("/predict", methods=["POST"])
def predict():

    # Check whether image was provided
    if "image" not in request.files:
        return jsonify({
            "error": "No image provided. Please upload an image."
        }), 400

    try:

        # Get uploaded image
        image_file = request.files["image"]

        # Open image and convert to RGB
        image = Image.open(image_file).convert("RGB")

        # Resize image to CIFAR-10 input size
        image = image.resize((32, 32))

        # Convert image to NumPy array
        image_array = np.array(image)

        # Normalize pixel values
        image_array = image_array.astype("float32") / 255.0

        # Add batch dimension
        image_array = np.expand_dims(image_array, axis=0)
```

```
        # Make prediction
        predictions = model.predict(
            image_array,
            verbose=0
        )

        # Find predicted class
        predicted_index = np.argmax(predictions[0])

        predicted_class = class_names[predicted_index]

        # Calculate confidence
        confidence = float(predictions[0][predicted_index]) * 100

        # Return JSON response
        return jsonify({
            "predicted_class": predicted_class,
            "confidence": round(confidence, 2)
        })

    except Exception as e:

        return jsonify({
            "error": str(e)
        }), 500


# ========================================
# Run Flask Application
# ========================================

if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=5000,
        debug=False
    )
```

Three design properties of `flask_api.py` matter for scaling. First, the model is loaded when the module is imported, before the server starts listening; a new pod therefore needs some time before port 5000 accepts connections. Second, the handler holds no per-user state, so replicas are interchangeable. Third, `debug=False` and `host="0.0.0.0"` make the server suitable for running inside a container.

Listing 6.7 – app.py (Streamlit front end)

```
import streamlit as st
import requests
from PIL import Image

# ========================================
# Page Configuration & Styling
# ========================================

st.set_page_config(
    page_title="CIFAR-10 Deep Learning App",
    page_icon="🧠",
    layout="wide"
)
```

```
# ========================================
# UI Styling
# ========================================

st.markdown(
    """
    <style>
    [data-testid="stMetricValue"] {
        font-size: 28px;
        font-weight: 700;
    }

    .class-badge {
        border: 1px solid #313745;
        border-radius: 8px;
        padding: 10px;
        text-align: center;
        font-weight: 600;
        margin-bottom: 10px;
    }

    .class-badge:hover {
        border-color: #4CAF50;
    }

    [data-testid="stHeader"] {
        display: none !important;
    }

    #MainMenu {
        visibility: hidden !important;
    }

    footer {
        visibility: hidden !important;
    }

    header {
        visibility: hidden !important;
    }

    .stApp > header {
        display: none !important;
    }

    button[title="View fullscreen"] {
        display: none !important;
    }

    [data-testid="stElementToolbar"] {
        display: none !important;
    }

    [data-testid="stStyledFullScreenButton"] {
        display: none !important;
    }

    [data-testid="stStatusWidget"] {
        display: none !important;
    }
    </style>
    """,
    unsafe_allow_html=True
)

# ========================================
# Flask API Configuration
```

```
# ==========================================

# IMPORTANT:
# "flask-api" will be the Flask service name
# inside docker-compose.yml.
FLASK_API_URL = "http://flask-api-service:5000"

# ==========================================
# CIFAR-10 Class Names
# ==========================================

class_names = [
    "Airplane ✈️",
    "Automobile 🚗",
    "Bird 🐦",
    "Cat 🐱",
    "Deer 🦌",
    "Dog 🐶",
    "Frog 🐸",
    "Horse 🐴",
    "Ship 🚢",
    "Truck 🚚"
]

# ==========================================
# Sidebar
# ==========================================

st.sidebar.title("🧠 CIFAR-10 AI Platform")
st.sidebar.markdown("---")

st.sidebar.info(
    "**Deep Learning Showcase**\n\n"
    "This application uses a Streamlit frontend "
    "connected to a Flask REST API for CIFAR-10 image classification."
)

st.sidebar.markdown("---")
st.sidebar.caption("⚡ Streamlit + Flask + TensorFlow + Docker")

# ==========================================
# Hero Banner
# ==========================================

st.title("🧠 CIFAR-10 Image Classification Suite")

st.caption(
    "Interactive Neural Network Dashboard & Flask API Integration"
)

st.markdown(
    """
    > **Overview:** Experience real-time multi-class object recognition
    using a Convolutional Neural Network (CNN) served through a Flask
    REST API and displayed through a Streamlit frontend.
    """
)

st.markdown("---")

# ==========================================
# Key Dataset Metrics
# ==========================================

st.subheader("📊 Key Overview Metrics")
```

```
m1, m2, m3, m4, m5 = st.columns(5)

with m1:
    st.metric(
        "Training Set",
        "50,000",
        help="Number of images used during model training"
    )

with m2:
    st.metric(
        "Testing Set",
        "10,000",
        help="Independent images used for evaluation"
    )

with m3:
    st.metric(
        "Resolution",
        "32 × 32 px",
        help="RGB Color Input Dimensions"
    )

with m4:
    st.metric(
        "Target Classes",
        "10 Categories"
    )

with m5:
    st.metric(
        "Test Accuracy",
        "72.4%",
        delta="Epochs: 10"
    )

st.markdown("---")

# ==========================================
# Class Categories
# ==========================================

st.subheader("🏷️ Target Classification Categories")

cols = st.columns(5)

for idx, class_name in enumerate(class_names):

    with cols[idx % 5]:

        st.markdown(
            f"""
            <div class="class-badge">
                Class {idx}<br>
                <b>{class_name}</b>
            </div>
            """,
            unsafe_allow_html=True
        )

st.markdown("---")

# ==========================================
# Neural Network Configuration
# ==========================================
```

```
st.subheader("🤖 Neural Network Configuration")

with st.expander(
    "🔍 Click to view detailed model architecture parameters",
    expanded=True
):

    col_a, col_b = st.columns(2)

    with col_a:

        st.markdown(
            """
            * **Architecture:** Sequential Convolutional Neural Network (CNN)
            * **Optimization Algorithm:** Adam
            * **Loss Calculation:** Sparse Categorical Crossentropy
            * **Classification Layer:** 10 Units with Softmax Activation
            """
        )

    with col_b:

        st.markdown(
            """
            * **Input Tensor Shape:** `(32, 32, 3)`
            * **Feature Extractors:** 2x Conv2D + MaxPooling Blocks
            * **Regularization:** Dropout (0.5 Rate)
            * **Hidden Layer Units:** 128 Dense Units (ReLU Activation)
            """
        )

st.markdown("---")

# ==========================================
# Flask API Connection Check
# ==========================================

st.subheader("🔌 Backend API Status")

if st.button("🔄 Check Flask API Connection"):

    try:

        response = requests.get(
            f"{FLASK_API_URL}/",
            timeout=5
        )

        if response.status_code == 200:

            st.success(
                "✅ Flask API is connected and running."
            )

            st.json(response.json())

        else:

            st.error(
                f"⚠️ Flask API returned status code "
                f"{response.status_code}"
            )

    except requests.exceptions.RequestException as e:
```

```
        st.error(
            "❌ Unable to connect to Flask API."
        )

        st.caption(
            f"Connection details: {e}"
        )

# ========================================
# Image Prediction
# ========================================

st.markdown("---")

st.subheader("🔮 Image Prediction")

st.write(
    "Upload an image and the Streamlit frontend will send it "
    "to the Flask REST API for prediction."
)

uploaded_file = st.file_uploader(
    "📤 Upload an image",
    type=["jpg", "jpeg", "png"]
)

if uploaded_file is not None:

    # Display uploaded image

    image = Image.open(uploaded_file).convert("RGB")

    col1, col2 = st.columns(2)

    with col1:

        st.image(
            image,
            caption="Uploaded Image",
            use_container_width=True
        )

    with col2:

        st.info(
            "Image received successfully. "
            "Click the button below to send it to the Flask API."
        )

        predict_button = st.button(
            "🚀 Predict Image",
            type="primary"
        )

        if predict_button:

            try:

                # Reset file position

                uploaded_file.seek(0)

                # Prepare image for Flask API

                files = {
```

```
        "image": (
            uploaded_file.name,
            uploaded_file,
            uploaded_file.type
        )
    }

    # Send image to Flask API

    response = requests.post(
        f"{FLASK_API_URL}/predict",
        files=files,
        timeout=60
    )

    # ========================================
    # Successful Response
    # ========================================

    if response.status_code == 200:

        result = response.json()

        predicted_class = result.get(
            "predicted_class",
            "Unknown"
        )

        confidence = result.get(
            "confidence",
            0
        )

        st.success(
            "✅ Prediction completed successfully!"
        )

        st.metric(
            "Predicted Class",
            predicted_class
        )

        st.metric(
            "Confidence",
            f"{confidence:.2f}%"
        )

        st.progress(
            min(max(confidence / 100, 0.0), 1.0)
        )

        st.json(result)

    # ========================================
    # Flask Error Response
    # ========================================

    else:

        try:
            error_data = response.json()
        except Exception:
            error_data = {
                "error": response.text
            }

        st.error(
```

```
                        f"❌ Flask API error "
                        f"(HTTP {response.status_code})"
                    )

                    st.json(error_data)

            except requests.exceptions.RequestException as e:

                st.error(
                    "❌ Could not connect to Flask API."
                )

                st.warning(
                    "Make sure the Flask container is running "
                    "and both containers are connected through "
                    "the Docker network."
                )

                st.caption(
                    f"Connection details: {e}"
                )

            except Exception as e:

                st.error(
                    "❌ An unexpected error occurred."
                )

                st.caption(
                    f"Error details: {e}"
                )

# ========================================
# Footer / Status
# ========================================

st.markdown("---")

st.caption(
    "🐬 Task 11 — Full Application Containerization | "
    "Streamlit Frontend → Flask REST API → CIFAR-10 CNN Model"
)
```

In `app.py` the line `FLASK_API_URL = "http://flask-api-service:5000"` is the link between the two deployments. It uses the Kubernetes service name rather than a pod address, so the front end keeps working when Flask pods are added, removed or replaced. The comment above the constant still mentions the Compose service name, a remnant of the earlier task, but the value itself is the Kubernetes service name.

7. Initial Deployment Verification

7.1 Purpose of the Baseline

Before scaling or updating anything, the existing state of the application was recorded. A baseline is essential for scaling and update experiments because every later claim of the form "the deployment changed from X to Y" requires a verified X. Three commands were used: one for deployments, one for pods with extended output and one for services.

```
kubectl get deployments

kubectl get pods -o wide

kubectl get services
```

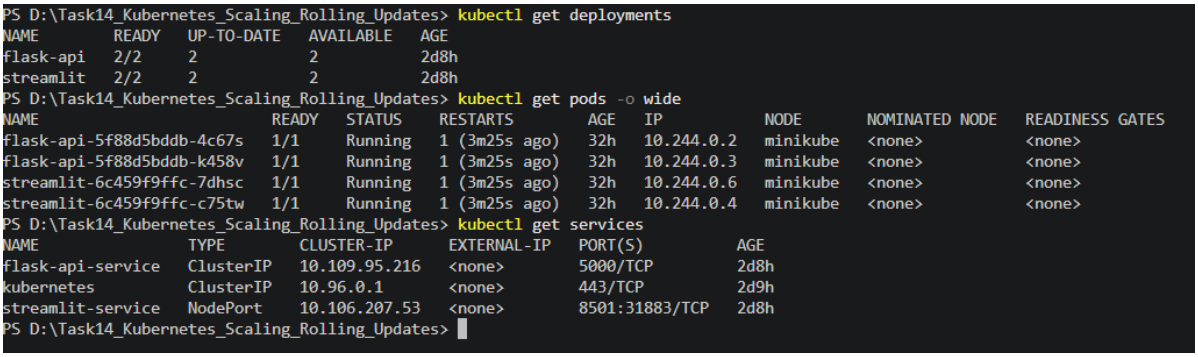


Figure 3: *Baseline state: deployments, pods with extended output and services before scaling*

7.2 Deployments

In Figure 3 both deployments report the same pattern: `flask-api` shows READY 2/2, UP-TO-DATE 2, AVAILABLE 2 and AGE 2d8h, and `streamlit` shows READY 2/2, UP-TO-DATE 2, AVAILABLE 2 and AGE 2d8h. The three number columns have distinct meanings. READY compares ready pods with the desired count, UP-TO-DATE counts pods that match the current pod template, and AVAILABLE counts pods that have been ready long enough to serve traffic. Equal values in all three columns mean that the desired state has been fully reached.

7.3 Pods

Pod	Ready	Status	Restarts	Age	Pod IP
flask-api-5f88d5bddb-4c67s	1/1	Running	1 (3m25s ago)	32h	10.244.0.2
flask-api-5f88d5bddb-k458v	1/1	Running	1 (3m25s ago)	32h	10.244.0.3
streamlit-6c459f9ffc-7dhsc	1/1	Running	1 (3m25s ago)	32h	10.244.0.6

Pod	Ready	Status	Restarts	Age	Pod IP
streamlit-6c459f9ffc-c75tw	1/1	Running	1 (3m25s ago)	32h	10.244.0.4

The pod names follow the pattern `<deployment>-<replicaset hash>-<random suffix>`. All Flask pods share the hash `5f88d5bddb` and all Streamlit pods share `6c459f9ffc`, which shows that each pair of pods belongs to a single ReplicaSet. This is the starting point for the later rolling update, because the update will create a Flask ReplicaSet with a different hash. All four pods are scheduled on the node `minikube` and each has its own IP address in the pod network range 10.244.0.0/24.

7.4 Services

Service	Type	Cluster IP	Port(s)	Age
flask-api-service	ClusterIP	10.109.95.216	5000/TCP	2d8h
kubernetes	ClusterIP	10.96.0.1	443/TCP	2d9h
streamlit-service	NodePort	10.106.207.53	8501:31883/TCP	2d8h

The Flask API is reachable only inside the cluster through a ClusterIP service on port 5000. The Streamlit front end is exposed through a NodePort service: port 8501 inside the cluster is mapped to node port 31883. The service named `kubernetes` is created automatically by Kubernetes and exposes the API server inside the cluster.

7.5 Streamlit Deployment Manifest

The Streamlit deployment, which remained unchanged during the whole task, is defined as follows:

Listing 7.1 – streamlit-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: streamlit
spec:
  replicas: 2
  selector:
    matchLabels:
      app: streamlit
  strategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: streamlit
    spec:
      containers:
```

```
      - name: streamlit
        image: task12-streamlit:latest
        imagePullPolicy: Never
        ports:
          - containerPort: 8501
        resources:
          requests:
            cpu: "250m"
            memory: "512Mi"
          limits:
            cpu: "500m"
            memory: "1Gi"
```

This manifest declares two replicas, the default `RollingUpdate` strategy (with Kubernetes default values for surge and unavailability because none are specified), the locally available image `task12-streamlit:latest` with `imagePullPolicy: Never`, and resource requests of 250m CPU and 512Mi memory with limits of 500m CPU and 1Gi memory.

7.6 Observations from the Baseline

- Both deployments were fully available: every desired replica was ready, up to date and available.
- The Flask API had two replicas, which is the starting value for the scaling operation.
- Pods already carried a restart count of one, recorded only a few minutes before the screenshot; this agrees with the cluster restart in Section 5 and did not affect readiness.
- The services were already in place, so the application path from Streamlit through `flask-api-service` to the Flask pods existed before any change.
- No Metrics Server was yet enabled, so resource usage could not yet be inspected; this is addressed in Section 11.

With the baseline established as two healthy Flask replicas and two healthy Streamlit replicas, the next step was to change the number of Flask replicas.

8. Application Scaling

8.1 What Scaling Means in Kubernetes

Scaling is the act of changing how many copies of an application run at the same time. Kubernetes supports two directions. **Vertical scaling** gives each copy more CPU or memory; in Kubernetes this corresponds to changing the resource requests and limits of a container and recreating the pod. **Horizontal scaling** changes the number of copies, called replicas, and is the form used in this task. Horizontal scaling is preferred for stateless services because each added replica is an independent, identical worker and the load can be spread across them.

8.2 The Objects Involved

Object	Responsibility in a scaling operation
Deployment	Holds the desired state: the pod template, the strategy and the desired replica count (`spec.replicas`).
ReplicaSet	Created by the Deployment for a specific pod template; keeps exactly the requested number of matching pods alive.
Pod	One running copy of the application; the unit that is added or removed when scaling.
Service	Provides a stable name and virtual IP and distributes connections to the ready pods that match its selector.
Scheduler and kubelet	Place the new pod on a node and start its container.

When the replica count of a Deployment is changed, the Deployment controller updates the ReplicaSet that is currently active. The ReplicaSet controller notices that the number of pods differs from the desired number and creates the missing pods. No new ReplicaSet is created, because the pod template has not changed. This distinction between scaling and updating is visible in the evidence of Section 9 and Section 14.

8.3 Why the Flask API Was Selected for Scaling

The Flask API performs the computationally expensive part of the application: image preprocessing and neural network inference. In a production setting, inference is the stage that becomes a bottleneck first, since a single process can handle only a limited number of predictions at a time. The Streamlit front end performs light work, mainly rendering the page and forwarding the image, and it remained at two replicas. The resource measurements in

Section 11 are consistent with this reasoning: a Flask pod uses roughly five times the memory of a Streamlit pod because it holds TensorFlow and the model in memory.

Scaling the Flask API also has a second benefit that is independent of load. With three replicas, the loss of one pod, for example during an update or after a crash, leaves two pods still able to answer requests. The Service continues to route requests to the remaining ready pods.

8.4 Two Ways to Scale

Method	Command or setting	Characteristics
Imperative	`kubectl scale deployment flask-api --replicas=3`	Immediate, one command, changes only the live object. Used in this task.
Declarative	`replicas: 3` in `flask-deployment.yaml` and `kubectl apply -f flask-deployment.yaml`	Recorded in a file that can be version-controlled; re-applying the file keeps the replica count at 3.
Automatic (not used)	HorizontalPodAutoscaler	Adjusts replicas from observed metrics; needs resource requests and a working Metrics Server.

The imperative command was used for the scaling step, and the manifest later used for the rolling update declares `replicas: 3`. The two are consistent: applying the manifest does not change the replica count back to two. This is an important practical point, because applying a manifest that still said `replicas: 2` after a manual scale would silently reduce the deployment again.

8.5 Preconditions Checked Before Scaling

- The deployment was healthy at 2/2 ready replicas (Section 7).
- The node was Ready and could accept more pods (Section 5).
- The pod template requests 250m CPU and 512Mi memory per Flask pod, so a third pod adds 250m CPU and 512Mi to the cluster's scheduling requests. The scheduler placed the new pod successfully, which shows that the node had enough allocatable capacity.
- The image `task12-flask-api:latest` was already present on the node, which is required because `imagePullPolicy` is `Never`.

8.6 Expected Outcome

Quantity	Before scaling	Expected after scaling
Desired Flask replicas	2	3

Quantity	Before scaling	Expected after scaling
Flask pods Running	2	3
Deployment READY column	2/2	3/3
New ReplicaSet created	–	No (template unchanged)
Existing Flask pods replaced	–	No; a third pod is only added
Streamlit replicas	2	2 (unchanged)

The next section carries out the operation and compares the outcome with this table.

9. Scaling from 2 to 3 Replicas

9.1 Executing the Scale Operation

The Flask API deployment was scaled with a single command and the outcome was checked immediately with two listing commands:

```
kubectl scale deployment flask-api --replicas=3
kubectl get deployments
kubectl get pods -o wide
```

```
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl scale deployment flask-api --replicas=3
deployment.apps/flask-api scaled
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl get deployments
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
flask-api   3/3     3            3           2d8h
streamlit   2/2     2            2           2d8h
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl get pods -o wide
NAME                         READY   STATUS    RESTARTS        AGE     IP           NODE       NOMINATED NODE   READINESS GATES
flask-api-5f88d5bddb-4c67s   1/1     Running   1 (8m45s ago)   33h     10.244.0.2   minikube   <none>           <none>
flask-api-5f88d5bddb-fc8r2   1/1     Running   0               2m30s   10.244.0.7   minikube   <none>           <none>
flask-api-5f88d5bddb-k458v   1/1     Running   1 (8m45s ago)   33h     10.244.0.3   minikube   <none>           <none>
streamlit-6c459f9ffc-7dhsc   1/1     Running   1 (8m45s ago)   33h     10.244.0.6   minikube   <none>           <none>
streamlit-6c459f9ffc-c75tw   1/1     Running   1 (8m45s ago)   33h     10.244.0.4   minikube   <none>           <none>
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates>
```

Figure 4: *Scaling the Flask API deployment to three replicas and listing deployments and pods*

The command returned `deployment.apps/flask-api scaled`, confirming that the API server accepted the change (Figure 4). The deployment listing then showed `flask-api` at READY 3/3, UP-TO-DATE 3 and AVAILABLE 3, while `streamlit` remained at 2/2.

9.2 Pod-Level Result

Pod	Restarts	Age	Pod IP	Note
flask-api-5f88d5bddb-4c67s	1 (8m45s ago)	33h	10.244.0.2	Existing pod
flask-api-5f88d5bddb-fc8r2	0	2m30s	10.244.0.7	New pod created by scaling
flask-api-5f88d5bddb-k458v	1 (8m45s ago)	33h	10.244.0.3	Existing pod
streamlit-6c459f9ffc-7dhsc	1 (8m45s ago)	33h	10.244.0.6	Unchanged
streamlit-6c459f9ffc-c75tw	1 (8m45s ago)	33h	10.244.0.4	Unchanged

Three observations follow directly from the table. First, the new pod `flask-api-5f88d5bddb-fc8r2` carries the same ReplicaSet hash as the two existing pods, which proves that scaling added a pod to the existing ReplicaSet instead of creating a new one. Second, the two original Flask pods kept their names, IP addresses and ages, so they were not disturbed.

Third, the new pod has zero restarts and is Running, 1/1 ready, with its own IP address 10.244.0.7.

9.3 Rollout Status Check

```
kubectl rollout status deployment/flask-api

kubectl get deployment flask-api
```

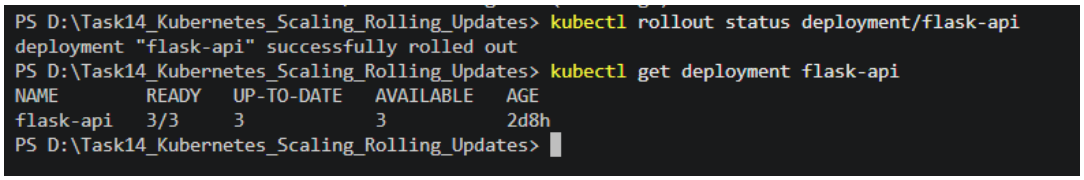


Figure 5: *Rollout status after scaling reports that the flask-api deployment was successfully rolled out*

Figure 5 shows the message `deployment "flask-api" successfully rolled out` followed by the deployment line `flask-api 3/3 3 3`. The rollout status command is normally associated with updates, but it is equally useful after scaling because it waits until the desired number of replicas is available and reports success only when that is true.

9.4 What Happened Inside the Cluster

1. `kubectl` sent a request to the API server to change the scale sub-resource of the `flask-api` Deployment from 2 to 3.
2. The API server stored the new desired replica count in the cluster database.
3. The Deployment controller propagated the new count to the active ReplicaSet `flask-api-5f88d5bddb`.
4. The ReplicaSet controller saw two pods where three were wanted and created one new pod object from the existing pod template.
5. The scheduler assigned the new pod to the only node, `minikube`.
6. The kubelet on that node started the container from the local image `task12-flask-api:latest`; the container loaded the model and began serving on port 5000.
7. Once the container was ready, the endpoints behind `flask-api-service` were updated so that the service could route traffic to the third pod as well.

9.5 Timing Evidence

The screenshots contain relative times that help to reconstruct the sequence. In the baseline screenshot the Flask pods had been restarted "3m25s ago". In the post-scaling listing the same

restart is reported as "8m45s ago", so about five minutes and twenty seconds of wall-clock time separated the two captures. The new pod is 2m30s old in the post-scaling listing, so it was created roughly three minutes after the baseline capture, which matches the manual steps of typing and running the scale command between the two screenshots. The pod reached the Running and 1/1 state by the time of the listing, and the table above shows no restart for it.

9.6 Comparison with the Expected Outcome

Quantity	Expected	Observed	Result
Flask deployment READY	3/3	3/3	Met
Flask pods Running	3	3	Met
New ReplicaSet created	No	No (same hash 5f88d5bddb)	Met
Existing Flask pods replaced	No	No (same names, IPs and ages)	Met
Streamlit replicas	2	2	Met
Rollout status	Success	successfully rolled out	Met

9.7 Effect on the Architecture

After scaling, requests to `flask-api-service:5000` can be served by any of three pods at the addresses 10.244.0.2, 10.244.0.3 and 10.244.0.7. From the point of view of Streamlit nothing changed: it keeps calling the same service name. This is the main advantage of placing a Service in front of a Deployment, because capacity changes behind the Service are invisible to the client.

The scaling step was completed with no downtime and no change to any file. It demonstrates the first evaluation criterion, scaling effectiveness: a single command increased capacity by one half, the new capacity was ready within minutes and the existing pods were not interrupted.

10. Pod and Deployment Monitoring

10.1 Monitoring Commands Used

Monitoring in this task relied on a small set of kubectl read commands. Each one answers a different question about the health of the application:

Command	Question answered	Used in
`kubectl get deployments`	Are the desired replicas ready, up to date and available?	Sections 7, 9, 10, 19
`kubectl get pods -o wide`	Which pods exist, are they Running, how often did they restart, which IP and node do they have?	Sections 7, 9, 10
`kubectl get pods`	Short pod listing for final verification.	Section 19
`kubectl get rs`	Which ReplicaSets exist and how many pods does each own?	Section 14
`kubectl rollout status deployment/flask-api`	Has the rollout finished?	Sections 9, 19
`kubectl rollout history deployment/flask-api`	Which revisions of the deployment are recorded?	Section 15
`kubectl get services`	Which services exist and how are they exposed?	Sections 7, 16
`kubectl top pods`	How much CPU and memory does each pod use?	Section 11

10.2 Deployment State After the Rolling Update

```
kubectl get deployments

kubectl get pods -o wide
```

```
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl get deployments
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
flask-api   3/3     3            3           2d9h
streamlit   2/2     2            2           2d9h
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl get pods -o wide
NAME                         READY   STATUS    RESTARTS      AGE     IP           NODE       NOMINATED NODE   READINESS GATES
flask-api-fd4b44db5-4dgpq    1/1     Running   0             5m13s   10.244.0.10  minikube   <none>           <none>
flask-api-fd4b44db5-k6j6j    1/1     Running   0             5m13s   10.244.0.11  minikube   <none>           <none>
flask-api-fd4b44db5-nss6s    1/1     Running   0             5m12s   10.244.0.12  minikube   <none>           <none>
streamlit-6c459f9ffc-7dhsc   1/1     Running   1 (41m ago)   33h     10.244.0.6   minikube   <none>           <none>
streamlit-6c459f9ffc-c75tw   1/1     Running   1 (41m ago)   33h     10.244.0.4   minikube   <none>           <none>
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates>
```

Figure 6: *Deployment and pod status after the rolling update*

Figure 6 shows the state of the system after the rolling update described in Section 13. Both deployments are fully available: `flask-api` is 3/3 and `streamlit` is 2/2, both with an age of 2d9h, which confirms that the Deployment objects themselves were updated in place and never deleted.

Pod	Ready	Status	Restarts	Age	Pod IP
flask-api-fd4b44db5-4dgpq	1/1	Running	0	5m13s	10.244.0.10
flask-api-fd4b44db5-k6j6j	1/1	Running	0	5m13s	10.244.0.11
flask-api-fd4b44db5-nss6s	1/1	Running	0	5m12s	10.244.0.12
streamlit-6c459f9ffc-7dhsc	1/1	Running	1 (41m ago)	33h	10.244.0.6
streamlit-6c459f9ffc-c75tw	1/1	Running	1 (41m ago)	33h	10.244.0.4

10.3 What Changed Between Baseline and Post-Update

Aspect	Before the update	After the update
Flask pod names	`flask-api-5f88d5bddb-*`	`flask-api-fd4b44db5-*`
Flask pod IPs	10.244.0.2, 10.244.0.3, 10.244.0.7	10.244.0.10, 10.244.0.11, 10.244.0.12
Flask pod ages	33h (two pods) and minutes (one pod)	about 5 minutes (all three)
Flask restarts	1 for the original pods, 0 for the new pod	0 for all three new pods
Streamlit pods	`streamlit-6c459f9ffc-*`	identical names, IPs and ages
Deployment ages	2d8h	2d9h (same object)

The table shows the signature of a successful rolling update: all pods that belong to the changed deployment have been replaced by new pods created from a new ReplicaSet, while the unchanged Streamlit deployment kept the same pods. The new Flask pods start with zero restarts, which indicates that the containers started cleanly and did not crash while loading the model.

10.4 Final Verification Snapshot

A later verification, shown in Figure 18 in Section 19, repeated the same listing about 14 minutes after the update. The Flask pods were still Running with zero restarts and the Streamlit pods were unchanged. The stability of the pod list between the two captures is itself an observation: there were no unexpected restarts or evictions in the period following the update.

10.5 Interpreting the Columns

- **READY** (for pods, for example 1/1) shows how many containers in the pod pass their readiness check. No explicit probes are defined in the manifests, so Kubernetes treats a container as ready as soon as it is running.

- **STATUS** `Running` means the pod has been bound to a node and its container has started. Other values that could appear in a problem situation include `Pending`, `ContainerCreating`, `CrashLoopBackOff` and `ImagePullBackOff`; none of them appeared in any screenshot.
- **RESTARTS** counts container restarts. A value of 0 for new pods and a small constant value for old pods indicates stability.
- **IP** is the pod IP, which changes whenever a pod is recreated. This is why the application uses a Service name rather than pod IPs.
- **NODE** is always `minikube` in this single-node cluster.

10.6 Monitoring Summary

Across all captures, no pod was in a state other than Running, no deployment had fewer ready pods than desired at the time of capture, and no new pod showed a restart. The monitoring evidence therefore supports the conclusion that the scaling and update operations did not destabilize the application.

11. Resource Utilization Monitoring

11.1 Enabling the Metrics Server

Kubernetes does not report CPU and memory usage by default. The Metrics Server must be installed so that the kubelets' resource readings are aggregated and exposed through the metrics API. In Minikube this is a single add-on command:

```
minikube addons enable metrics-server
```

Three follow-up commands verified that the add-on was working before measurements were taken:

```
kubectl get pods -n kube-system | findstr metrics
kubectl get apiservice v1beta1.metrics.k8s.io
kubectl top pods
```

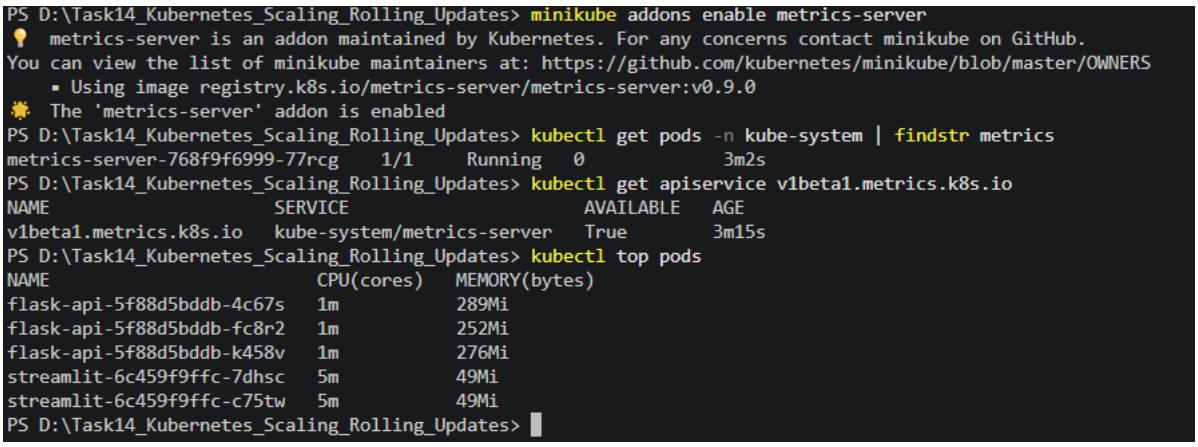


Figure 7: *Enabling the Metrics Server, verifying it and reading pod resource usage after scaling*

11.2 Verification of the Add-on

Check	Observed result	Meaning
Add-on enable output	Image `registry.k8s.io/metrics-server/metrics-server:v0.9.0`; "The 'metrics-server' addon is enabled"	The add-on was deployed from the stated image.
Pod in kube-system	`metrics-server-768f9f6999-77rcg` 1/1 Running, 0 restarts, age 3m2s	The Metrics Server pod itself was healthy.
APIService	`v1beta1.metrics.k8s.io` served by `kube-system/metrics-server`, AVAILABLE True	The metrics API was registered and reachable through the API server.

The `findstr` filter in the second command is the Windows PowerShell counterpart of `grep`. The `AVAILABLE True` value for the APIService is the key readiness signal: until it is True, `kubectl top` fails with an error stating that the metrics API is not available.

11.3 Measurements After Scaling

Pod	CPU (cores)	Memory
flask-api-5f88d5bddb-4c67s	1m	289Mi
flask-api-5f88d5bddb-fc8r2	1m	252Mi
flask-api-5f88d5bddb-k458v	1m	276Mi
streamlit-6c459f9ffc-7dhsc	5m	49Mi
streamlit-6c459f9ffc-c75tw	5m	49Mi

CPU is reported in millicores, where 1000m equals one CPU core, and memory in mebibytes. The three Flask replicas used between 252Mi and 289Mi each, for a total of 817Mi, and 1m CPU each. The two Streamlit replicas used 49Mi and 5m CPU each. The README quotes 252Mi for all three Flask pods; the screenshot shows slightly different values for the two older pods, so the screenshot values are used here. The older pods have been running longer and have served more activity, which may explain the higher memory of two of them, although the screenshots alone cannot prove the cause.

11.4 Measurements After the Rolling Update

```
kubectl top pods
```

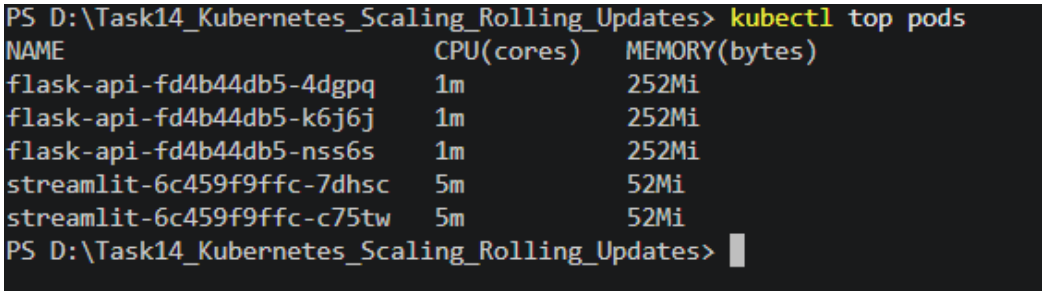


Figure 8: *Resource utilization immediately after the rolling update*

Pod	CPU (cores)	Memory
flask-api-fd4b44db5-4dgpq	1m	252Mi
flask-api-fd4b44db5-k6j6j	1m	252Mi
flask-api-fd4b44db5-nss6s	1m	252Mi
streamlit-6c459f9ffc-7dhsc	5m	52Mi

Pod	CPU (cores)	Memory
streamlit-6c459f9ffc-c75tw	5m	52Mi

Figure 8 shows that all three new Flask pods used exactly 1m CPU and 252Mi memory. The identical figures are expected for freshly started pods that run the same image and have not yet served varied requests. The total Flask memory was 756Mi, slightly less than the 817Mi measured before the update, because the new pods start from a clean state.

11.5 Final Measurements

The final measurement, shown in Figure 19 in Section 19, is the last screenshot in the sequence (numbered after the frog prediction screenshot). The Flask pods showed 1m CPU with 252Mi, 252Mi and 274Mi, and the Streamlit pods showed 5m and 6m CPU with 53Mi and 67Mi. The small rises in memory for one Flask pod (from 252Mi to 274Mi) and one Streamlit pod (from 52Mi to 67Mi) are consistent with those pods having handled the prediction request and page session.

11.6 Comparison of the Three Snapshots

Pod group	After scaling	After rolling update	Final check
Flask CPU per pod	1m, 1m, 1m	1m, 1m, 1m	1m, 1m, 1m
Flask memory per pod	289Mi, 252Mi, 276Mi	252Mi, 252Mi, 252Mi	252Mi, 252Mi, 274Mi
Flask memory total	817Mi	756Mi	778Mi
Streamlit CPU per pod	5m, 5m	5m, 5m	5m, 6m
Streamlit memory per pod	49Mi, 49Mi	52Mi, 52Mi	53Mi, 67Mi
Streamlit memory total	98Mi	104Mi	120Mi

11.7 Usage Compared with Requests and Limits

Both deployments declare the same resource settings: a request of 250m CPU and 512Mi memory, and a limit of 500m CPU and 1Gi memory. The request is the amount the scheduler reserves for the pod; the limit is the ceiling enforced at run time (CPU is throttled above the limit, and a container that exceeds its memory limit is terminated). The table compares the highest observed values with these settings.

Measure	Flask (highest observed)	Streamlit (highest observed)
CPU used vs 250m request	1m = 0.4%	6m = 2.4%

Measure	Flask (highest observed)	Streamlit (highest observed)
CPU used vs 500m limit	1m = 0.2%	6m = 1.2%
Memory used vs 512Mi request	289Mi = 56.4%	67Mi = 13.1%
Memory used vs 1Gi limit	289Mi = 28.2%	67Mi = 6.5%

All observed values are far below the limits, so there was no throttling and no risk of out-of-memory termination in the observed period. Memory is the more significant resource for the Flask pods: about half of the memory request and about one quarter of the limit is in use even when the API is idle. Because Kubernetes reserves the request, three Flask pods reserve 750m CPU and 1536Mi memory, although they use only a few millicores and about 780Mi. The CPU request is therefore generous compared with idle usage. This does not harm the demonstration, but a production deployment would tune the request after measuring real load, since an over-large request reduces the number of pods that fit on a node.

The measurements were taken while the API was idle or serving single test requests. They describe the baseline footprint of each replica, not the CPU cost of heavy inference. They are nevertheless valuable because they show that the footprint of a replica is stable and that adding a third replica adds a predictable amount of memory (about 250Mi to 290Mi).

11.8 Resource Observations

- Each Flask replica has a fixed memory footprint of roughly 250Mi, mostly attributable to the TensorFlow runtime since the model itself has only about 316 thousand parameters.
- Idle CPU usage of the Flask replicas is negligible (1m).
- Streamlit replicas use more CPU than idle Flask replicas (5m to 6m) because Streamlit keeps a script session alive, and less memory (49Mi to 67Mi).
- Resource usage was stable across the scaling and the rolling update; no replica approached its limits.

12. Rolling Update Configuration

12.1 Deployment Strategies in Kubernetes

A Deployment supports two update strategies. With **Recreate**, all old pods are terminated before any new pod is created; this is simple but causes downtime. With **RollingUpdate**, which is the default, old pods are replaced gradually: Kubernetes starts some new pods, waits for them to become available, and then stops some old pods, repeating until all pods run the new version. Rolling updates are the standard way to change a running service without interrupting it, and they are the focus of this task.

12.2 The Flask Deployment Manifest

The complete manifest used for the update is shown below and is also visible in the editor screenshot that follows it.

Listing 12.1 – flask-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: flask-api

spec:
  replicas: 3

  selector:
    matchLabels:
      app: flask-api

  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1

  template:
    metadata:
      labels:
        app: flask-api

    spec:
      containers:
        - name: flask-api
          image: task12-flask-api:latest
          imagePullPolicy: Never

          ports:
            - containerPort: 5000

          env:
            - name: TASK14_VERSION
              value: "v2"
```

```
        resources:
          requests:
            cpu: "250m"
            memory: "512Mi"

          limits:
            cpu: "500m"
            memory: "1Gi"
```

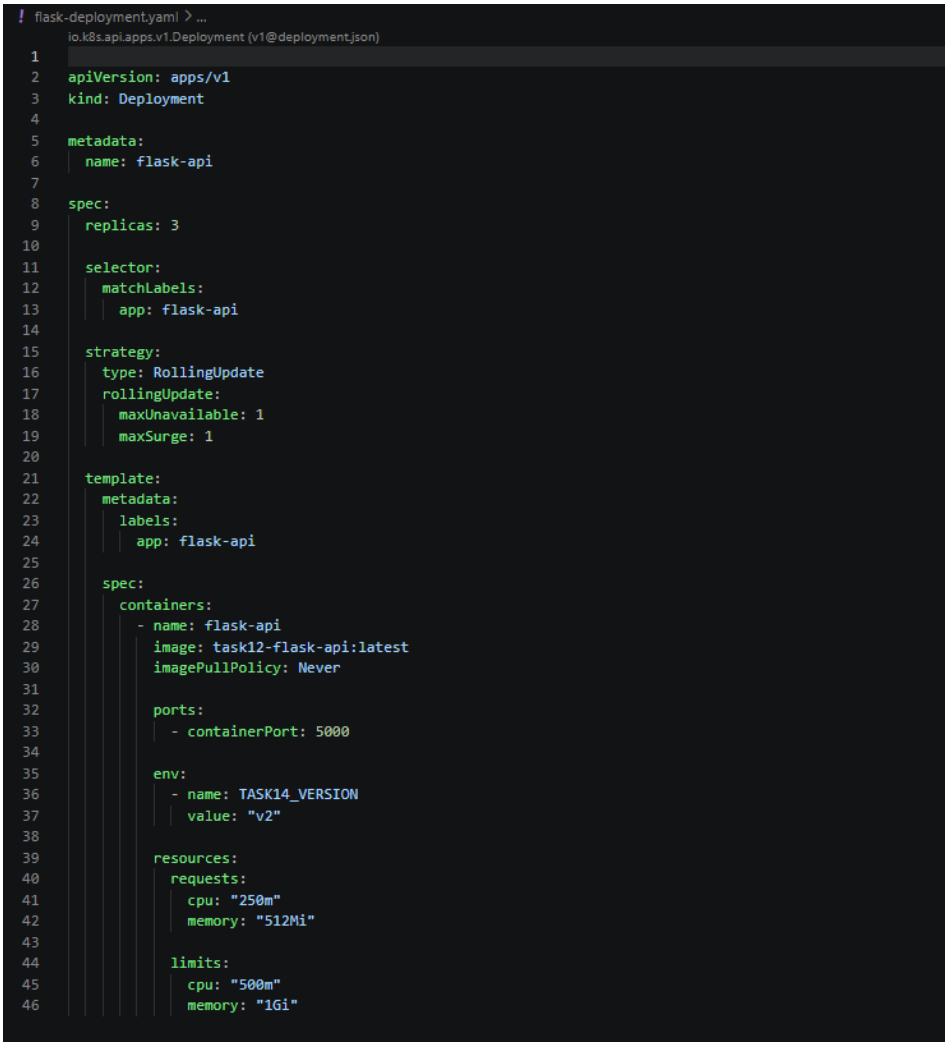


Figure 9: *The flask-deployment.yaml file opened in the editor, showing the RollingUpdate configuration*

12.3 Explanation of the Configuration

Field	Value	Explanation
`metadata.name`	`flask-api`	Name of the Deployment; also the prefix of pod and ReplicaSet names.
`spec.replicas`	3	Desired number of Flask pods; matches the result of the scaling step.

Field	Value	Explanation
`spec.selector.matchLabels`	`app: flask-api`	Labels by which the Deployment finds the pods it owns; must match the template labels.
`strategy.type`	`RollingUpdate`	Replace pods gradually instead of all at once.
`rollingUpdate.maxUnavailable`	1	At most one pod below the desired count may be unavailable at any time.
`rollingUpdate.maxSurge`	1	At most one pod above the desired count may exist at any time.
`template.metadata.labels`	`app: flask-api`	Label attached to pods; the Service selector uses the same label.
`image`	`task12-flask-api:latest`	Locally available image of the Flask API.
`imagePullPolicy`	`Never`	Kubernetes must not pull the image from a registry; it uses the copy on the node.
`ports.containerPort`	5000	Port on which Flask listens.
`env TASK14_VERSION`	`"v2"`	Environment variable added to force a pod template change.
`resources.requests`	250m CPU, 512Mi	Capacity reserved for each pod by the scheduler.
`resources.limits`	500m CPU, 1Gi	Maximum CPU and memory a container may use.

12.4 Effect of maxUnavailable and maxSurge

These two settings define the envelope within which the rollout may move. With three desired replicas, `maxUnavailable: 1` means that the number of available pods must never fall below two, and `maxSurge: 1` means that the total number of pods (old and new together) must never exceed four. Both values may be given as absolute numbers or as percentages of the desired count; this task uses absolute numbers.

Setting	Calculation with 3 replicas	Resulting bound
`maxUnavailable: 1`	3 desired − 1	At least 2 pods available at all times
`maxSurge: 1`	3 desired + 1	At most 4 pods exist at any moment
Combined	new pods and old pods together	Between 2 available and 4 total during the rollout

These settings represent a balanced choice. If both were zero, no progress would be possible. If `maxUnavailable` were 0 and `maxSurge` were 1, the service would keep full capacity

during the update but would need room for one extra pod. If `maxUnavailable` were 3, the strategy would behave almost like Recreate. The chosen values keep at least two-thirds of the capacity in service and need room for only one extra pod, which is appropriate for a single-node cluster.

12.5 Possible Rollout Sequences

Kubernetes may combine scale-up of the new ReplicaSet and scale-down of the old one in different orders depending on how quickly the new pods become available. One valid sequence under the chosen settings is shown below (O = old pod, N = new pod).

Step	Old pods	New pods	Total	Available (minimum)
Start	3	0	3	3
Surge: create one new pod	3	1	4	3
New pod ready; stop one old pod and create another new pod	2	2	4	2 or more
Continue replacing	1	3	4	2 or more
Finish	0	3	3	3

In every row the total stays at four or below and the available count stays at two or above, so the strategy limits are never violated.

12.6 Why an Environment Variable Was Used to Trigger the Update

A Deployment creates a new ReplicaSet only when something in its **pod template** changes, for example the image, the environment, the command or the labels. Changes outside the template, such as the replica count, do not start a rollout. In Task 14 the same image `task12-flask-api:latest` is used before and after the update, so a change of the image tag was not available as a trigger. Adding the environment variable `TASK14_VERSION` with the value `"v2"` changes the pod template without changing any application code, and therefore produces a real rolling update with no functional side effects. The variable is not read by `flask_api.py`; its only purpose is to mark the new revision.

12.7 Resource Settings and Quality of Service

Because the requests (250m CPU, 512Mi) are lower than the limits (500m CPU, 1Gi), Kubernetes assigns the pods the **Burstable** quality-of-service class. Burstable pods are guaranteed their requests and may use more up to the limits when the node has spare capacity.

During a rollout the scheduler must be able to place up to four Flask pods, which means 1000m CPU and 2048Mi memory of requests for Flask alone. The successful rollout shows that the node had sufficient allocatable capacity for this peak, in addition to the two Streamlit pods.

12.8 Configuration Summary

- Strategy: RollingUpdate with maxUnavailable 1 and maxSurge 1.
- Replicas: 3 (declared in the manifest, consistent with the scaling step).
- Trigger: pod-template change through the environment variable `TASK14_VERSION: "v2"`.
- Image: unchanged (`task12-flask-api:latest`, `imagePullPolicy: Never`).
- Resources: unchanged requests and limits, so the update tests only the rollout mechanism.

13. Rolling Update Implementation

13.1 Applying the Updated Manifest

The rolling update was started by applying the edited manifest to the cluster:

```
kubectl apply -f flask-deployment.yaml
```

The `apply` command compares the manifest with the live object. Because the only difference lies in the pod template (the new `TASK14_VERSION` entry), the Deployment controller treats the change as a new revision and starts a rollout. The Deployment object itself is updated in place, which is why its age stays at 2d9h in later listings.

13.2 Monitoring the Rollout

```
kubectl rollout status deployment/flask-api
```

The status command blocks and prints progress until the rollout is complete. The final line, visible in Figure 19, is `deployment "flask-api" successfully rolled out`. In an update that fails, for example because the new pods never become ready, this command would not report success within the progress deadline.

13.3 Sequence of Events

1. The API server stored the new pod template in the Deployment.
2. The Deployment controller computed a hash of the new template (`fd4b44db5`) and created a new ReplicaSet named `flask-api-fd4b44db5`.
3. Within the limits maxSurge 1 and maxUnavailable 1, the controller scaled the new ReplicaSet up and the old ReplicaSet `flask-api-5f88d5bddb` down in steps.
4. For each new pod the scheduler selected the node `minikube` and the kubelet started the container; the container loaded the model and began listening on port 5000.
5. As new pods became ready, the Service started to send traffic to them; as old pods were terminated, they were removed from the Service endpoints.
6. When three new pods were ready and zero old pods remained, the rollout was complete and the old ReplicaSet stayed in the cluster with zero replicas, ready to be used for a rollback.

13.4 Evidence of Completion

Evidence	Observation	Where shown
New pods	Three pods named `flask-api-fd4b44db5-*` created within one second of each other (ages 5m13s, 5m13s and 5m12s) at 10.244.0.10, .11 and .12	Figure 6
New ReplicaSet	`flask-api-fd4b44db5` with DESIRED 3, CURRENT 3, READY 3 and AGE 85s	Figure 10
Old ReplicaSets	`flask-api-5f88d5bddb` and `flask-api-56bcd9ffd7` at 0/0/0	Figure 10
History	Revisions 1, 2 and 3 recorded	Figure 11
Rollout status	`successfully rolled out`	Figure 19
Application behaviour	Service answered with its JSON message and Streamlit predicted Frog with 99.01% confidence	Figure 14, Figure 17

The three new pods were created within a second of each other according to their ages. This reflects the fact that the old and new ReplicaSets moved in quick succession: the cluster has one node, the image is local and there is no registry pull, so each new pod starts as soon as it is scheduled. In a larger cluster with remote images the steps would be spaced further apart.

13.5 Availability During the Update

The rollout limits guarantee that at least two Flask pods were available during the whole update. The README states that the update was performed without interrupting application availability, and the final tests after the update (Sections 17 and 18) show that the service and the application worked. It is worth being precise about what the evidence shows. The screenshots prove that the service answered correctly after the update; they do not include a continuous request loop running during the rollout. The guarantee for the period during the rollout therefore rests on the configured strategy (at least two available pods) rather than on a recorded stream of successful requests. Section 19 lists continuous probing during a rollout as a further improvement.

13.6 Impact on the Streamlit Deployment

The Streamlit Deployment was not modified. Its pods kept the names `streamlit-6c459f9ffc-7dhsc` and `streamlit-6c459f9ffc-c75tw`, the same IP addresses and the same ages, as shown in the post-update listings. The Streamlit page continued to call `flask-`

`api-service` and was served new Flask pods transparently after the update. This confirms that the Service abstraction isolates the front end from replacement of back-end pods.

13.7 Role of Readiness

Kubernetes decides that a new pod is ready to receive traffic on the basis of its readiness status. The manifests do not define a readiness probe, so a pod is treated as ready once its container has started. For the Flask API this is a small caveat: the container starts the Python process, which imports TensorFlow and loads the model before the server begins to listen. During that short period, a pod that is already counted as ready would refuse connections. In this single-node, local-image setup the window is short and no failure was observed in the evidence, but a readiness probe on the `/` endpoint would close this gap and make rolling updates safer. This is noted again in the recommendations.

13.8 Implementation Summary

The rolling update was implemented with one edit to the manifest and one `kubectl apply` command. Kubernetes created a new ReplicaSet, replaced all three Flask pods within the configured limits, retained the old ReplicaSets for history and reported the rollout as successful. This fulfils the second evaluation criterion, rolling update implementation.

14. ReplicaSet Verification

14.1 Role of ReplicaSets in Updates

A Deployment does not manage pods directly. It manages ReplicaSets, and each ReplicaSet manages the pods of one particular pod template. When the template changes, the Deployment creates a new ReplicaSet and gradually moves pods from the old ReplicaSet to the new one. Inspecting the ReplicaSets after an update is therefore the most direct way to verify that a rolling update took place.

```
kubectl get rs
```

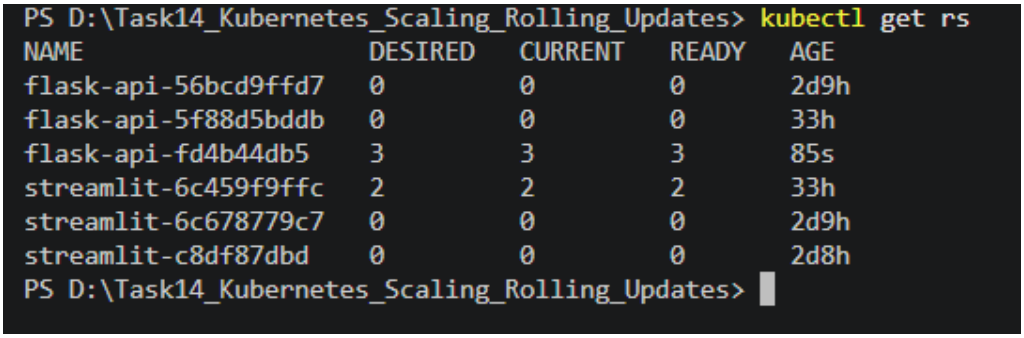


Figure 10: *ReplicaSets of the flask-api and streamlit deployments after the rolling update*

14.2 Observed ReplicaSets

ReplicaSet	Desired	Current	Ready	Age	Belongs to
flask-api-56bcd9ffd7	0	0	0	2d9h	flask-api (oldest revision)
flask-api-5f88d5bddb	0	0	0	33h	flask-api (previous revision)
flask-api-fd4b44db5	3	3	3	85s	flask-api (current revision)
streamlit-6c459f9ffc	2	2	2	33h	streamlit (current)
streamlit-6c678779c7	0	0	0	2d9h	streamlit (older)
streamlit-c8df87dbd	0	0	0	2d8h	streamlit (older)

14.3 Interpretation

- **The current Flask ReplicaSet is `flask-api-fd4b44db5`.** It has three desired, three current and three ready pods, and an age of only 85 seconds at the time of the screenshot, which matches the time of the `kubectl apply`.
- **Two older Flask ReplicaSets exist with zero pods.** `flask-api-5f88d5bddb` is the ReplicaSet that owned the pods before the update (Section 7 and Section 9), and

`flask-api-56bcd9ffd7` is an even older revision of the Deployment. Neither owns pods any more, but both remain as history.

- **The Streamlit deployment has one active ReplicaSet** (`streamlit-6c459f9ffc`, two pods) and two inactive ones, which suggests that its template was changed in earlier stages of the project but not during Task 14.
- **Only one ReplicaSet per Deployment is active.** The sum of the READY column for Flask is 3, the desired replica count, and for Streamlit is 2.

14.4 Why Old ReplicaSets Are Kept

Kubernetes keeps scaled-down ReplicaSets so that a Deployment can be rolled back quickly. A rollback does not need to rebuild anything: the old ReplicaSet is scaled up again and the new one is scaled down, again within maxSurge and maxUnavailable. The number of retained ReplicaSets is controlled by the `revisionHistoryLimit` field of the Deployment, which defaults to ten and was not changed in the manifests used here.

14.5 Linking Pod Names to ReplicaSets

Pod name prefix	ReplicaSet	State
`flask-api-5f88d5bddb-`	`flask-api-5f88d5bddb`	Pods before the update (4c67s, k458v, fc8r2); all terminated by the rollout
`flask-api-fd4b44db5-`	`flask-api-fd4b44db5`	Pods after the update (4dgpq, k6j6j, nss6s); Running
`streamlit-6c459f9ffc-`	`streamlit-6c459f9ffc`	Unchanged pods (7dhsc, c75tw); Running

The match between the hash in each pod name and the ReplicaSet name is exact in every row, so the ReplicaSet listing and the pod listing give a consistent picture of the cluster.

14.6 Verification Result

The ReplicaSet listing confirms that Kubernetes performed the update by moving the Flask pods from the previous ReplicaSet to a newly created one. The previous ReplicaSets were scaled down to zero and the latest ReplicaSet has three ready replicas. The rolling update mechanism therefore behaved exactly as designed.

15. Rollout History and Status

15.1 Revision History

Every time the pod template of a Deployment changes, Kubernetes records a new revision. The history is read with:

```
kubectl rollout history deployment/flask-api
```

```
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl rollout history deployment/flask-api
deployment.apps/flask-api
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>

PS D:\Task14_Kubernetes_Scaling_Rolling_Updates>
```

Figure 11: *Rollout history of the flask-api deployment*

The output in Figure 11 lists three revisions, numbered 1, 2 and 3, each with the CHANGE-CAUSE shown as `<none>`. Three revisions agree with the three Flask ReplicaSets seen in Section 14: one revision per ReplicaSet. Since the ReplicaSets are ordered by age (2d9h, 33h and 85s), revision 3 corresponds to the newest ReplicaSet, `flask-api-fd4b44db5`, created by the Task 14 update. This ordering is an inference from the ages; the history command does not print the ReplicaSet names directly.

15.2 Understanding CHANGE-CAUSE

The CHANGE-CAUSE column is filled from the annotation `kubernetes.io/change-cause` on the Deployment. It is empty here because no annotation was set. This does not affect the correctness of the rollout; it only means that the history does not describe why each revision was made. A good practice is to set the annotation after each significant change, for example:

```
kubectl annotate deployment/flask-api kubernetes.io/change-cause="Task
14 rolling update: TASK14_VERSION=v2"
```

This command was not run in the task and is mentioned only as a documentation improvement.

15.3 Commands Related to Rollout History

Command	Purpose	Used in this task
`kubectl rollout status deployment/flask-api`	Wait for and report the completion of a rollout.	Yes

Command	Purpose	Used in this task
`kubectl rollout history deployment/flask-api`	List the recorded revisions.	Yes
`kubectl rollout history deployment/flask-api --revision=3`	Show the pod template stored for a specific revision.	No
`kubectl rollout undo deployment/flask-api`	Return to the previous revision.	No
`kubectl rollout undo deployment/flask-api --to-revision=2`	Return to a chosen revision.	No
`kubectl rollout pause` and `resume`	Hold a rollout part-way and continue later.	No

15.4 Rollback Capability

Although no rollback was necessary, the history demonstrates that one is available. With three recorded revisions and two scaled-down ReplicaSets that still exist, running `kubectl rollout undo deployment/flask-api` would scale up the previous ReplicaSet and scale down the current one using the same RollingUpdate limits. In a failed update, this is the fastest route back to the last good state.

15.5 Combined Status Evidence

The final rollout status in Figure 19 reports that the deployment was successfully rolled out, and the final deployment listing in Figure 18 shows `flask-api` 3/3 and `streamlit` 2/2. Together with the history, these results show that the update is complete, recorded and reversible.

15.6 Timeline of Deployment Changes in Task 14

Order	Action	Effect on Flask deployment	Effect on revisions
1	`minikube start` and baseline checks	No change; 2/2 ready	None
2	`kubectl scale ... --replicas=3`	3/3 ready; one pod added to ReplicaSet `5f88d5bddb`	No new revision
3	Metrics Server enabled; `kubectl top pods`	No change to the Deployment	None
4	Edit manifest, `kubectl apply -f flask-deployment.yaml`	New ReplicaSet `fd4b44db5`; all pods replaced	New revision (3)
5	Verification of ReplicaSets, history, services and application	No change; 3/3 ready	None

Scaling does not change the template and so produces no new revision; the manifest change does. This distinction explains why the scaled pod belongs to the old ReplicaSet while the updated pods belong to a new one.

16. Service Verification

16.1 Why Services Matter

Pods are temporary. Each time a pod is replaced, as happened to all Flask pods during the rolling update, its name and IP address change. A Kubernetes **Service** gives a group of pods a stable name and a stable virtual IP. Clients connect to the Service, and Kubernetes forwards each connection to one of the ready pods whose labels match the Service selector. The set of pods behind a Service is tracked automatically, which is what makes scaling and rolling updates transparent to clients.

16.2 Service Manifests

Listing 16.1 – flask-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: flask-api-service
spec:
  selector:
    app: flask-api
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  type: ClusterIP
```

Listing 16.2 – streamlit-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: streamlit-service
spec:
  selector:
    app: streamlit
  ports:
    - protocol: TCP
      port: 8501
      targetPort: 8501
  type: NodePort
```

Property	flask-api-service	streamlit-service
Type	ClusterIP	NodePort
Selector	`app: flask-api`	`app: streamlit`
Service port	5000	8501
Target port (container)	5000	8501

Property	flask-api-service	streamlit-service
Reachable from	Inside the cluster only	Outside the cluster through a node port
Used by	Streamlit pods and the curl test pod	The user's browser

The selector of each Service matches the label that the corresponding Deployment places on its pods. The Flask Deployment template carries `app: flask-api`, which is the label selected by `flask-api-service`. Because the label is part of the template and did not change during the update, the new pods were picked up by the Service automatically.

16.3 Verification Command and Output

```
kubectl get services
```

```
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
flask-api-service   ClusterIP   10.109.95.216   <none>        5000/TCP         2d9h
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP          2d10h
streamlit-service   NodePort    10.106.207.53   <none>        8501:31883/TCP   2d9h
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates>
```

Figure 12: *Services after the rolling update*

Service	Type	Cluster IP	External IP	Ports	Age
flask-api-service	ClusterIP	10.109.95.216	<none>	5000/TCP	2d9h
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	2d10h
streamlit-service	NodePort	10.106.207.53	<none>	8501:31883/TCP	2d9h

The output in Figure 12 is identical in IP addresses and ports to the baseline in Section 7. Only the ages differ, increased by about an hour. This is the central result of the service check: although all three Flask pods were replaced, the Flask Service kept the same Cluster IP, `10.109.95.216`, and the same port, 5000. Clients that use the Service address or its DNS name were unaffected.

16.4 Service Discovery by Name

Inside the cluster, the DNS name `flask-api-service` resolves to the Cluster IP of the Service. The Streamlit application relies on this name through the line `FLASK_API_URL = "http://flask-api-service:5000"`. The curl test pod used in Section 17 uses the same

URL, so a successful response from the test pod demonstrates that the name resolves, that the Service forwards to a ready Flask pod and that the Flask application is running.

16.5 Exposure of the Streamlit Application

The Streamlit service is of type NodePort, which opens a port (31883) on the node and forwards it to port 8501 of the Streamlit pods. With the Minikube Docker driver on Windows, the node runs inside a Docker container, so the node port is not directly reachable from the browser on the host. The README therefore records the use of the Minikube helper:

```
minikube service streamlit-service --url
```

This command creates a local tunnel and prints a URL on the loopback address. The browser screenshot in Section 18 shows exactly such an address, `127.0.0.1:50635`, which is the tunnelled entry to the NodePort service. The high port number is chosen by Minikube for the tunnel and differs from the node port 31883.

16.6 Load Distribution Behind the Service

When more than one pod matches the selector, the Service distributes new connections among the ready pods. In the standard setup this is done by the `kube-proxy` component, which programs the node's networking rules so that each new connection to the Cluster IP is directed to one of the pod IPs. The distribution is per connection and not guaranteed to be perfectly even. With three Flask replicas, the Service has three possible destinations (10.244.0.10, 10.244.0.11 and 10.244.0.12 after the update). This is what turns the larger replica count into actual additional serving capacity.

16.7 Verification Result

- Both application services exist and have the expected types and ports.
- The Cluster IPs and ports did not change during scaling or the rolling update.
- The Flask service is internal only, which is appropriate because only the Streamlit front end needs to call it.
- The Streamlit service is exposed and was opened successfully in the browser.

17. Application Availability Testing

17.1 Test Method

Availability of the Flask API was tested from inside the cluster, which is the same network position as the Streamlit pods. A temporary pod was started from the `curlimages/curl` image, asked to request the health endpoint of the Flask service and then removed automatically. The test was run twice: once after the scaling step and once after the rolling update.

```
kubectl run flask-test --rm -it --restart=Never --image=curlimages/curl
-- curl -s http://flask-api-service:5000/
```

Part of the command	Meaning
`kubectl run flask-test`	Create a pod named `flask-test`.
`--rm`	Delete the pod when it exits.
`-it`	Attach an interactive terminal so the output is shown.
`--restart=Never`	Run the pod once; do not restart it after it finishes.
`--image=curlimages/curl`	Use a small image that contains the curl tool.
`-- curl -s http://flask-api-service:5000/`	Call the health endpoint through the Service name in silent mode.

17.2 Test 1 – After Scaling to Three Replicas

```
kubectl get service flask-api-service
```

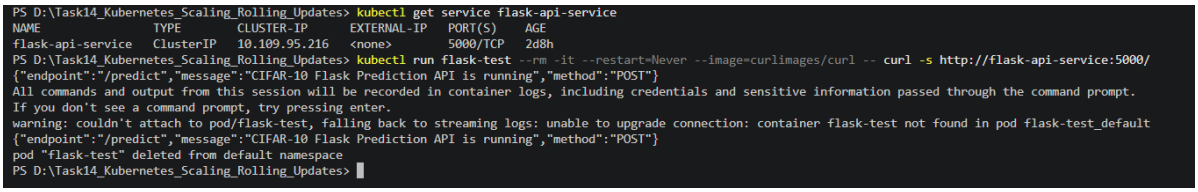


Figure 13: *Service check and in-cluster curl test after scaling the Flask API to three replicas*

The first command in Figure 13 confirms that `flask-api-service` is a ClusterIP service with the address 10.109.95.216 on port 5000. The curl test then returned the following JSON, which is exactly the response that `flask_api.py` produces for the `/` route:

Response received

```
{"endpoint":"/predict","message":"CIFAR-10 Flask Prediction API is
running","method":"POST"}
```

The output appears twice in the screenshot. The reason is that the `-it` option tried to attach to a container that had already finished (curl exits immediately). The warning line `couldn't attach to pod/flask-test, falling back to streaming logs` explains that kubectl then fetched the output from the pod logs instead. The first copy of the JSON comes from the initial attach and the second from the log fallback. The security notice that precedes it is a standard message from the container image and not an error. The last line, `pod "flask-test" deleted from default namespace`, confirms that `--rm` cleaned up the temporary pod.

17.3 Test 2 – After the Rolling Update

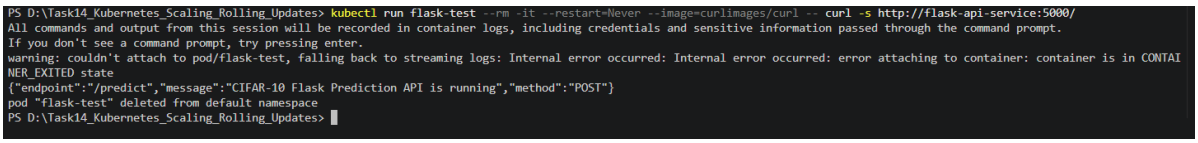


Figure 14: *In-cluster curl test after the rolling update*

The test was repeated after the rolling update (Figure 14) with the same command. The Flask service returned the same JSON message. The warning text differs slightly from the first test: it states that an internal error occurred while attaching because the container was in the `CONTAINER_EXITED` state. This is again the attach step failing for a very short-lived container, followed by the fallback to logs, which printed the response. The important result is the JSON line itself and the final deletion of the pod.

17.4 Comparison of the Two Tests

Aspect	After scaling	After rolling update
Flask pods serving	3 (pod hash 5f88d5bddb)	3 (pod hash fd4b44db5)
Service address	10.109.95.216:5000	10.109.95.216:5000 (unchanged)
Command	Identical	Identical
Response	JSON: Flask Prediction API is running	JSON: Flask Prediction API is running
Attach warning	Container not found in pod	Container in CONTAINER_EXITED state
Result	Available	Available

17.5 What the Tests Prove and What They Do Not

The tests prove that, at the moment of each request, the DNS name resolved, the Service forwarded the connection to a ready Flask pod and the Flask application produced a correct response. Because the same command gave the same result before and after the update, the update did not break service discovery or the application.

A single request cannot prove which replica answered or that all three replicas respond, since the Service selects one pod per connection. Proving that every replica serves traffic would require repeated requests and the inspection of pod logs; this was not part of the recorded evidence and is therefore not claimed here. Similarly, the tests do not measure latency or throughput.

17.6 Availability of the Complete Path

The curl test covers the back end only. The complete path, from the browser through Streamlit and the Flask Service to the model, is tested in Section 18 by uploading an actual image. Together the two tests cover both the internal service path and the user-facing path.

17.7 Summary

- The Flask service was reachable by name from inside the cluster after scaling and after the rolling update.
- The response content was correct and identical in both tests.
- Temporary test pods were removed automatically, leaving the cluster in its previous state.

18. Streamlit Application Verification

18.1 Accessing the Application

The Streamlit front end was opened in a web browser through the Minikube service tunnel. The browser address bar in the screenshot shows `127.0.0.1:50635`, the local tunnel to `streamlit-service`. The page loaded fully, which shows that at least one Streamlit pod was running and the NodePort service forwarded traffic correctly.

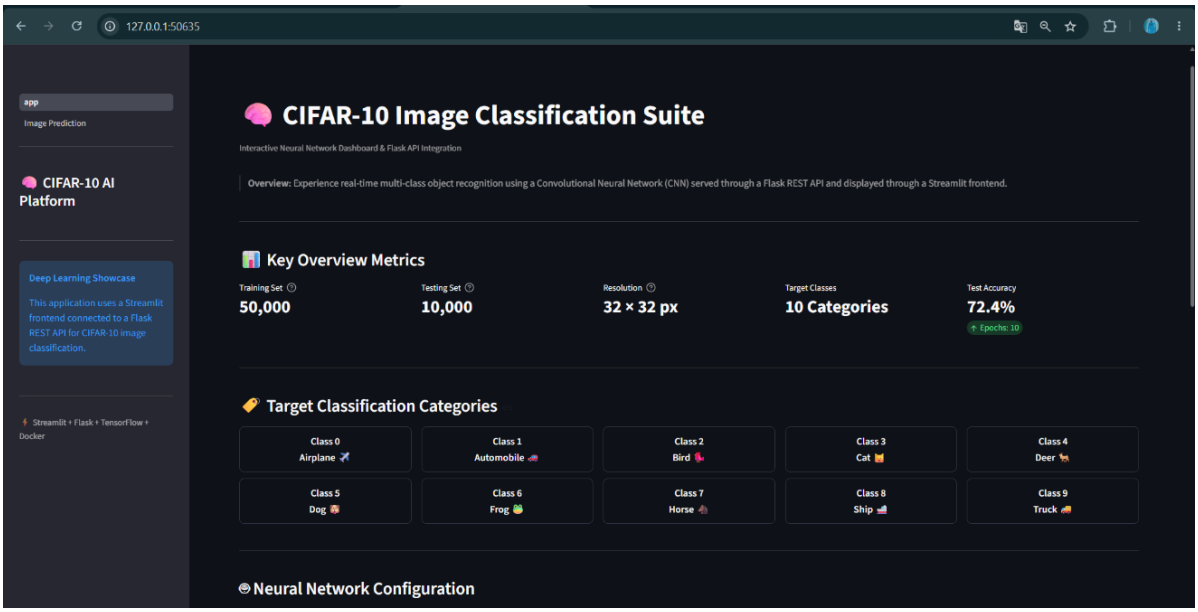


Figure 15: *The Streamlit application opened in the browser through the Minikube service URL*

18.2 Content of the Dashboard

Figure 15 shows the top of the dashboard generated by `app.py`. The sidebar offers two pages, `app` and `Image Prediction`, followed by the title "CIFAR-10 AI Platform" and an information panel describing the Streamlit and Flask integration. The main area shows the title "CIFAR-10 Image Classification Suite" with a one-line subtitle and an overview text, a row of key metrics, and a grid of the ten target classes.

Dashboard metric	Value displayed
Training set	50,000
Testing set	10,000
Resolution	32 × 32 px
Target classes	10 Categories
Test accuracy	72.4% (Epochs: 10)

The class grid lists Class 0 to Class 9 in the order Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck. This order is the same as the `class_names` list in `flask_api.py`, which is necessary for the index returned by the model to map to the correct label. Frog is class 6.

18.3 Test Input

The test image was a photograph of a green frog on a white background, saved as `Frog.png` (216.4 KB according to the uploader). The image is shown below as it was uploaded. Note that the image is far larger than the 32 × 32 pixel input expected by the model; the Flask API reduces it to the required size.



Figure 16: *Frog image used for the prediction test*

18.4 Prediction After the Update

After the rolling update, the image was uploaded in the Streamlit page and the **Predict Image** button was pressed. The Streamlit script sent the file to `http://flask-api-service:5000/predict`; the request was handled by one of the new Flask pods and the result was returned and displayed.

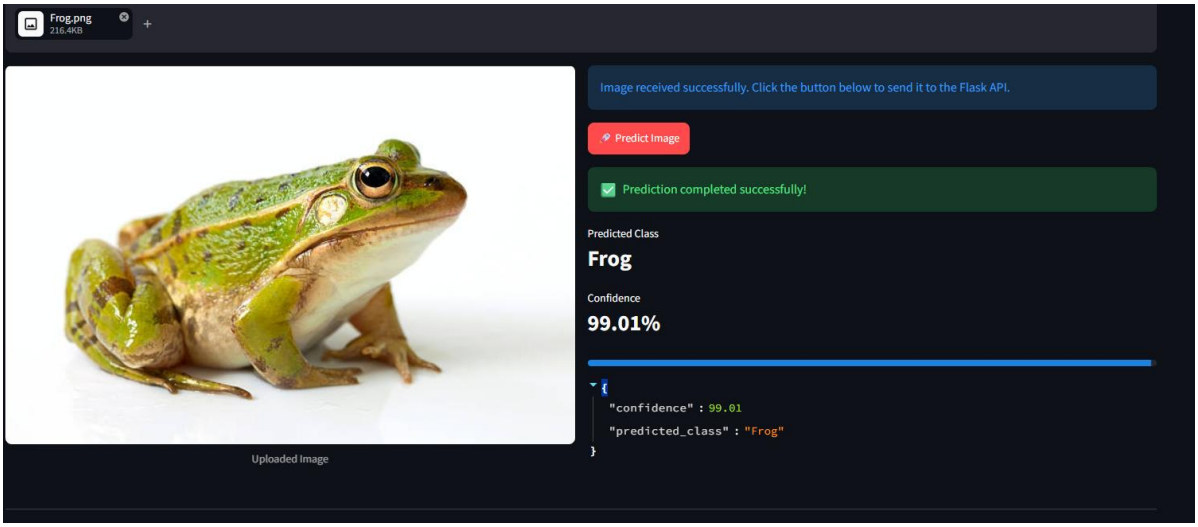


Figure 17: *Prediction result for the frog image after the rolling update*

Item	Value
Uploaded file	Frog.png (216.4KB)
Message shown	Prediction completed successfully!
Predicted Class	Frog
Confidence	99.01%
Raw JSON returned	{"confidence": 99.01, "predicted_class": "Frog"}

18.5 Interpretation

The model returned the correct class with a confidence of 99.01%. The result confirms that the entire chain functioned after the update: the browser reached Streamlit, Streamlit reached the Flask service by name, the service reached a Flask pod from the new ReplicaSet, the model inside that pod was loaded and produced a prediction, and the JSON travelled back to the browser.

Two clarifications are useful for a careful reading. The confidence value is the highest softmax probability for this single image; it is not the accuracy of the model, which the dashboard gives as 72.4% on the test set. Also, a single correct prediction proves that the workflow works, not that the model is accurate on all images. In Task 14 the aim of the test is functional verification of the deployment, and for that purpose one successful end-to-end prediction is the appropriate evidence.

18.6 Streamlit Code Path Exercised

- The image was opened with Pillow and shown on the left of the page (`st.image`).
- On pressing the button, the file pointer was reset with `uploaded_file.seek(0)` and the file was posted as the multipart field `image`.
- The response status 200 led to the success branch, which displayed the class with `st.metric`, the confidence formatted to two decimals, a progress bar scaled from the percentage and the raw JSON with `st.json`.
- No error branch (HTTP error or connection error) was triggered, which is consistent with the screenshot showing only the success message.

18.7 Summary

The Streamlit application opened successfully, the Flask API responded to a real prediction request and the correct class was returned after the rolling update. This verifies that the scaling and update operations did not affect the user-facing behaviour of the application.

19. Results, Observations and Analysis

19.1 Final Kubernetes Verification

At the end of the task, the deployments, pods and services were listed once more, and the resource usage and rollout status were recorded.

```
kubectl get deployments

kubectl get pods

kubectl get services
```

```
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl get deployments
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
flask-api   3/3     3            3           2d9h
streamlit   2/2     2            2           2d9h
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl get pods
NAME                         READY   STATUS    RESTARTS      AGE
flask-api-fd4b44db5-4dgpq    1/1     Running   0             14m
flask-api-fd4b44db5-k6j6j    1/1     Running   0             14m
flask-api-fd4b44db5-nss6s    1/1     Running   0             14m
streamlit-6c459f9ffc-7dhsc   1/1     Running   1 (50m ago)   33h
streamlit-6c459f9ffc-c75tw   1/1     Running   1 (50m ago)   33h
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
flask-api-service   ClusterIP   10.109.95.216   <none>        5000/TCP         2d9h
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP          2d10h
streamlit-service   NodePort    10.106.207.53   <none>        8501:31883/TCP   2d9h
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates>
```

Figure 18: *Final verification of deployments, pods and services*

Figure 18 shows that `flask-api` is 3/3 and `streamlit` is 2/2, all five pods are Running and 1/1 ready, the Flask pods are about 14 minutes old with zero restarts, and the services are unchanged.

```
kubectl top pods

kubectl rollout status deployment/flask-api
```

```
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl top pods
NAME                         CPU(cores)   MEMORY(bytes)
flask-api-fd4b44db5-4dgpq    1m           252Mi
flask-api-fd4b44db5-k6j6j    1m           252Mi
flask-api-fd4b44db5-nss6s    1m           274Mi
streamlit-6c459f9ffc-7dhsc   5m           53Mi
streamlit-6c459f9ffc-c75tw   6m           67Mi
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates> kubectl rollout status deployment/flask-api
deployment "flask-api" successfully rolled out
PS D:\Task14_Kubernetes_Scaling_Rolling_Updates>
```

Figure 19: *Final resource utilization and rollout status*

Figure 19 shows final CPU and memory figures for all pods and the closing message that the deployment was successfully rolled out.

19.2 Summary of Results

Stage	Flask replicas	Streamlit replicas	Observation
Baseline	2/2	2/2	All pods Running; services in place
After scaling	3/3	2/2	One pod added to the existing ReplicaSet; rollout successful
After rolling update	3/3 (new ReplicaSet)	2/2	Three new pods; old ReplicaSets scaled to 0; history shows 3 revisions
Final verification	3/3	2/2	Pods Running with zero restarts; services unchanged; rollout successful

19.3 Scaling Effectiveness

- The scale command increased the Flask deployment from two to three ready replicas with a single command.
- The additional pod was scheduled and running within minutes, without restarting or recreating the existing pods.
- The pod was added to the existing ReplicaSet, proving that scaling is distinct from updating.
- The Service kept the same address, so the extra capacity became available to clients without any configuration change.
- Each replica adds a predictable resource cost of about 250 to 290Mi of memory and a negligible idle CPU load.

The scaling objective was therefore met. The observation that scaling increases capacity must be read with the single-node limitation in mind: all replicas share the same node and its CPU, so the benefit of the third replica for heavy load depends on the node having spare CPU capacity. The measured idle CPU use of 1m suggests that there was ample spare capacity.

19.4 Rolling Update Implementation

- The `RollingUpdate` strategy with `maxUnavailable: 1` and `maxSurge: 1` was configured explicitly in the manifest.
- A change in the pod template triggered a new ReplicaSet, `flask-api-fd4b44db5`.
- All three Flask pods were replaced; the new pods are Running with zero restarts.
- Old ReplicaSets were retained at zero replicas and the history recorded three revisions.

- The Streamlit deployment was left untouched and kept its pods.

19.5 System Stability

- No pod was in a failed, pending or restarting state in any screenshot taken after the update.
- Restart counters of the new Flask pods remained at zero over at least the 14 minutes between the update and the final check.
- CPU and memory usage stayed far below limits at every measurement.
- The Flask service answered correctly before and after the update, and the Streamlit prediction produced the right class with 99.01% confidence.

19.6 Performance Observations

Observation	Value	Comment
Flask memory per pod	252Mi to 289Mi	Stable; dominated by the TensorFlow runtime.
Flask CPU per pod	1m	Idle or near idle.
Streamlit memory per pod	49Mi to 67Mi	Rises slightly after an interactive session.
Streamlit CPU per pod	5m to 6m	Small but higher than idle Flask.
Highest memory vs limit	289Mi of 1Gi (28.2%)	Wide safety margin.
Highest CPU vs limit	6m of 500m (1.2%)	No throttling risk.
Prediction confidence	99.01% for Frog	Single functional test, not an accuracy measure.

19.7 Evaluation Against Task Criteria

Criterion	Assessment	Supporting evidence
Scaling effectiveness	Achieved: 2 to 3 replicas, all ready, no disruption to existing pods.	Figure 3, Figure 4, Figure 5
Rolling update implementation	Achieved: configured strategy, new ReplicaSet, old ones scaled down, history recorded.	Figure 9, Figure 10, Figure 11
System stability	Achieved: Running pods, no new restarts, low resource usage, working service and prediction.	Figure 6, Figure 8, Figure 14, Figure 17

19.8 Deliverables Checklist

Deliverable	Provided as
Scaling documentation	Sections 8 and 9, with the manifests and commands in Sections 12 and 13
Monitoring screenshots	Seventeen screenshots, embedded as figures throughout the report
Performance observations	Section 11 (resource measurements) and Section 19.6
Source code	Listings in Sections 6, 7, 12 and 16
Outputs, results and observations	Sections 7 to 19

19.9 Limitations

- **Single node.** All replicas run on one node, so scaling increases process-level redundancy but not protection against the failure of the node.
- **No load test.** The resource figures describe idle and light use. The effect of heavy concurrent prediction traffic on CPU was not measured.
- **No probes.** Neither manifest defines readiness or liveness probes, so readiness is inferred from container start rather than from a successful health check.
- **Availability during the rollout was not recorded continuously.** The evidence shows correct behaviour after the update, and the strategy guarantees at least two available pods during it.
- **Empty change cause.** The history does not record why each revision was created.

19.10 Recommendations

1. Add a readiness probe on `GET /` and a liveness probe to the Flask container, so that traffic is sent only to pods that have loaded the model.
2. Pin library versions in `requirements.txt` and use versioned image tags instead of `latest`, so that each revision refers to a reproducible image.
3. Annotate each deployment change with `kubernetes.io/change-cause` to make the rollout history self-explanatory.
4. Add a HorizontalPodAutoscaler for the Flask deployment, using the Metrics Server that is now enabled, so that the replica count follows the load automatically.
5. Tune resource requests after a load test; the observed idle CPU of 1m is far below the 250m request.

6. Run the cluster with more than one node if protection against node failure is required.
7. Record a continuous request loop during a rollout to document availability during the update itself.

20. Conclusion

20.1 Summary of Work

Task 14 applied Kubernetes scaling and rolling update capabilities to a containerized CIFAR-10 deep learning application consisting of a Flask prediction API and a Streamlit front end. A Minikube cluster with the Docker driver was started and verified. The baseline deployment of two Flask and two Streamlit replicas was recorded. The Flask API was scaled from two to three replicas with `kubectl scale`, the Metrics Server was enabled and the resource consumption of every pod was measured. A rolling update with `maxUnavailable: 1` and `maxSurge: 1` was configured and triggered through a pod-template change; the resulting ReplicaSets, rollout history and rollout status were verified. The Flask service and the Streamlit application were tested after the update.

20.2 Key Findings

- Scaling the Flask deployment added a pod to the existing ReplicaSet and left existing pods untouched, producing 3/3 ready replicas.
- The rolling update replaced all three Flask pods through a new ReplicaSet, `flask-api-fd4b44db5`, and left the previous ReplicaSets at zero replicas for rollback.
- The Flask Service kept its Cluster IP and port throughout, which kept the Streamlit front end and the curl test working without changes.
- Each Flask pod uses about 250Mi to 290Mi of memory and 1m CPU when idle, well below the configured limits.
- After the update, the Streamlit application correctly classified the frog image with 99.01% confidence.

20.3 Conclusion

The evidence demonstrates that Kubernetes can change the capacity of the application and replace its pods in a controlled way while keeping the service reachable. The three evaluation criteria, scaling effectiveness, rolling update implementation and system stability, are all supported by the screenshots and outputs presented in this report. The task also exposes the practical points that matter in real deployments: the value of a stable Service address, the difference between scaling and updating, the importance of resource measurements and the

need for readiness probes and versioned images. With these improvements in place, the same approach could be extended to automatic scaling and multi-node clusters.